SourceCraft

Анализ кода в эпоху ИИ: **это конец?**

Андрей Кулешов,
руководитель SourceCraft Security

$ whoami?

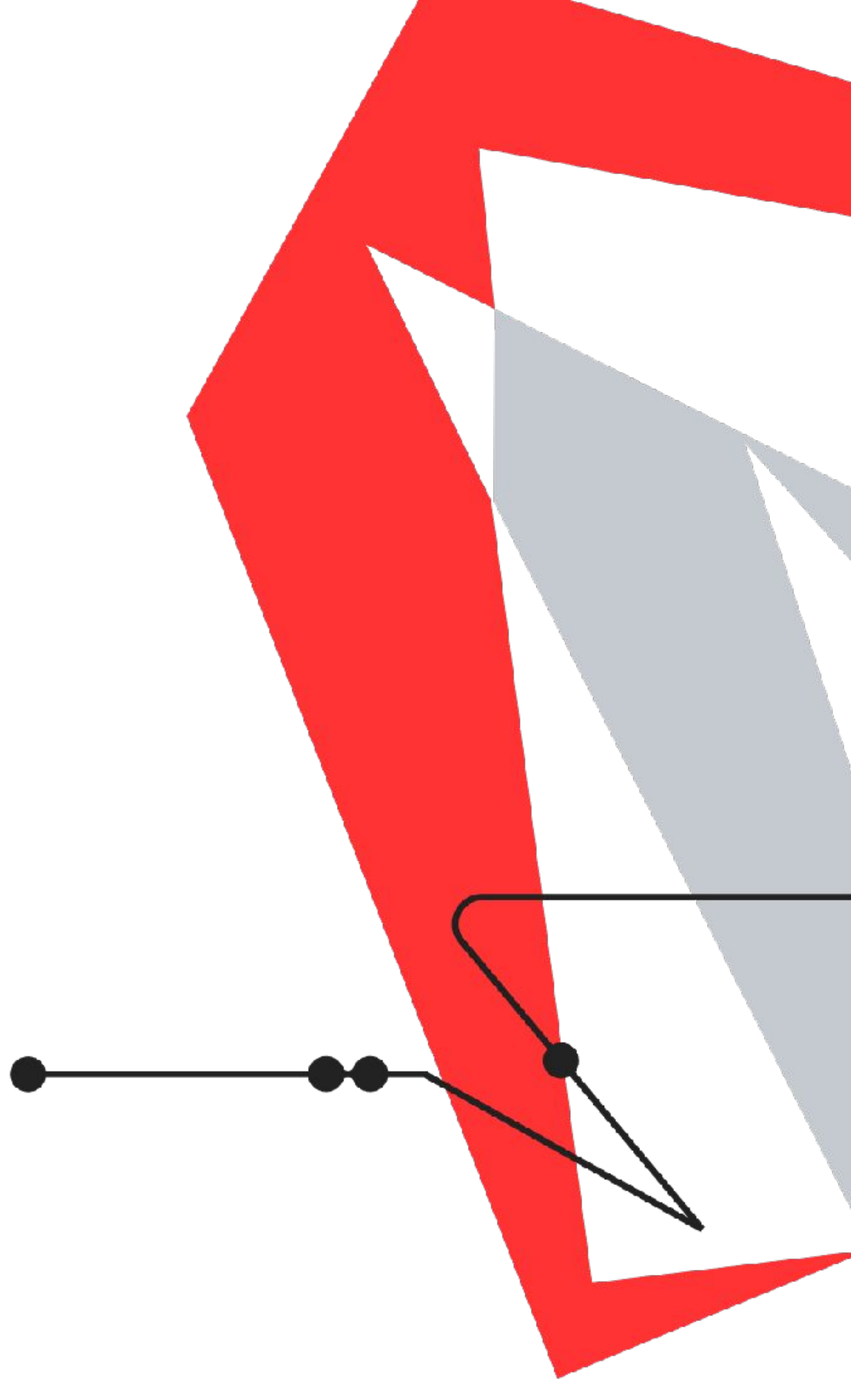


Андрей Кулешов

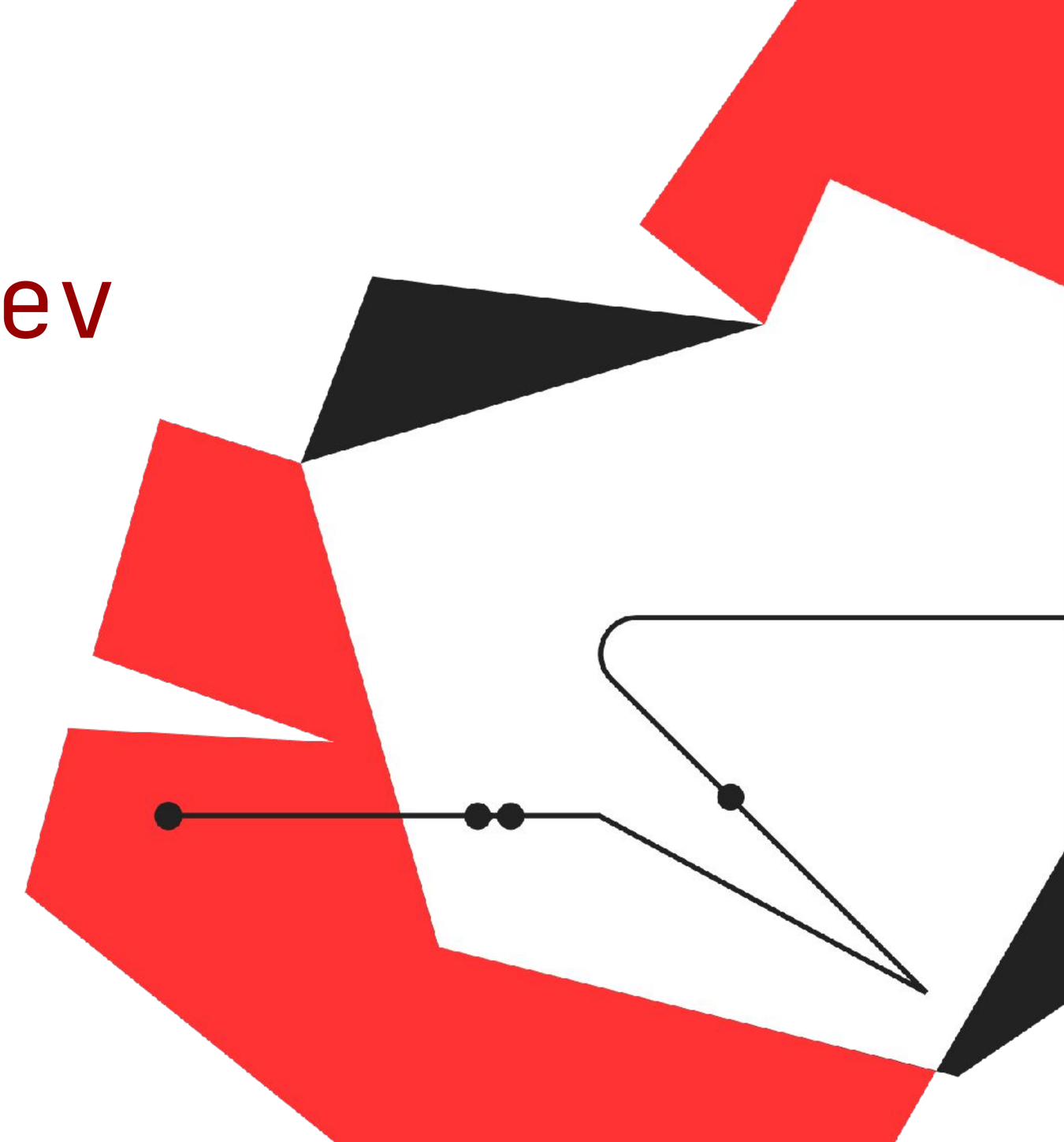
Руководитель разработки
SourceCraft Security


Это конец

Но давайте все-таки обсудим: **а чем мы тогда занимаемся?**



Платформа для разработчиков SourceCraft.dev



. AI-native платформа для разработки
. Глубоко интегрирована с Yandex Cloud
. Конкурент экосистем GitHub и GitLab

Но ключевое отличие в том, что мы начали создавать эту платформу уже в эпоху ИИ: **без ограничений, связанных с legacy**, которые могли бы нас сдерживать, и с **мощной поддержкой широко используемоего Облака от Яндекса**.

Add DAST in uploader

Created: 23 Jul 2026 • Updated: 23 Jul 2026 • 4

Stepan merged **4 commits** from `feature/axidex/uploader-sync` into main

Overview | Files changed 6 | Commits 4 | +88 -15

Pull request successfully merged and closed
The source branch was automatically deleted after the merge.

All required checks have passed
4 Success

Description

Generate with AI PRO

What's done:

- Added DAST
- Removed SA

Linked Issues

Add issue

Activity

Add your markup here

Change request | Draft | Comment

Выглядит как обычная платформа для разработки

Author
Stepan

Reviewers
All approvals received
Andrey Kuleshov
Auto assign

Code review rules
All passed
Details

Labels
Add label

appsec
Public repository

Search

Plan
- Issues
- Milestones
- Labels

Code
- Browse
- Pull requests
- Branches
- Commits
- Tags
- Releases
- Spaces NEW

Automations
- CI/CD
- AI skills
- Webhooks

Security
- Overview
- Secrets scanning
- Dependencies
- Code scanning

Repository settings

Add DAST in uploader

Created: 23 Jul 2026 • Updated: 23 Jul 2026 • 4

Stepan merged **4 commits** from `feature/axidex/uploader-sync` into main

Overview | Files changed 6 | Commits 4 | +88 -15

Pull request successfully merged and closed
The source branch was automatically deleted after the merge.

All required checks have passed
4 Success

Description

Generate with AI PRO

What's done:

- Added DAST
- Removed SA

Linked Issues

Add issue

Activity

Add your markup here

Change request | Draft | Comment

Author
Stepan

Reviewers
All approvals received
Andrey Kuleshov
Auto assign

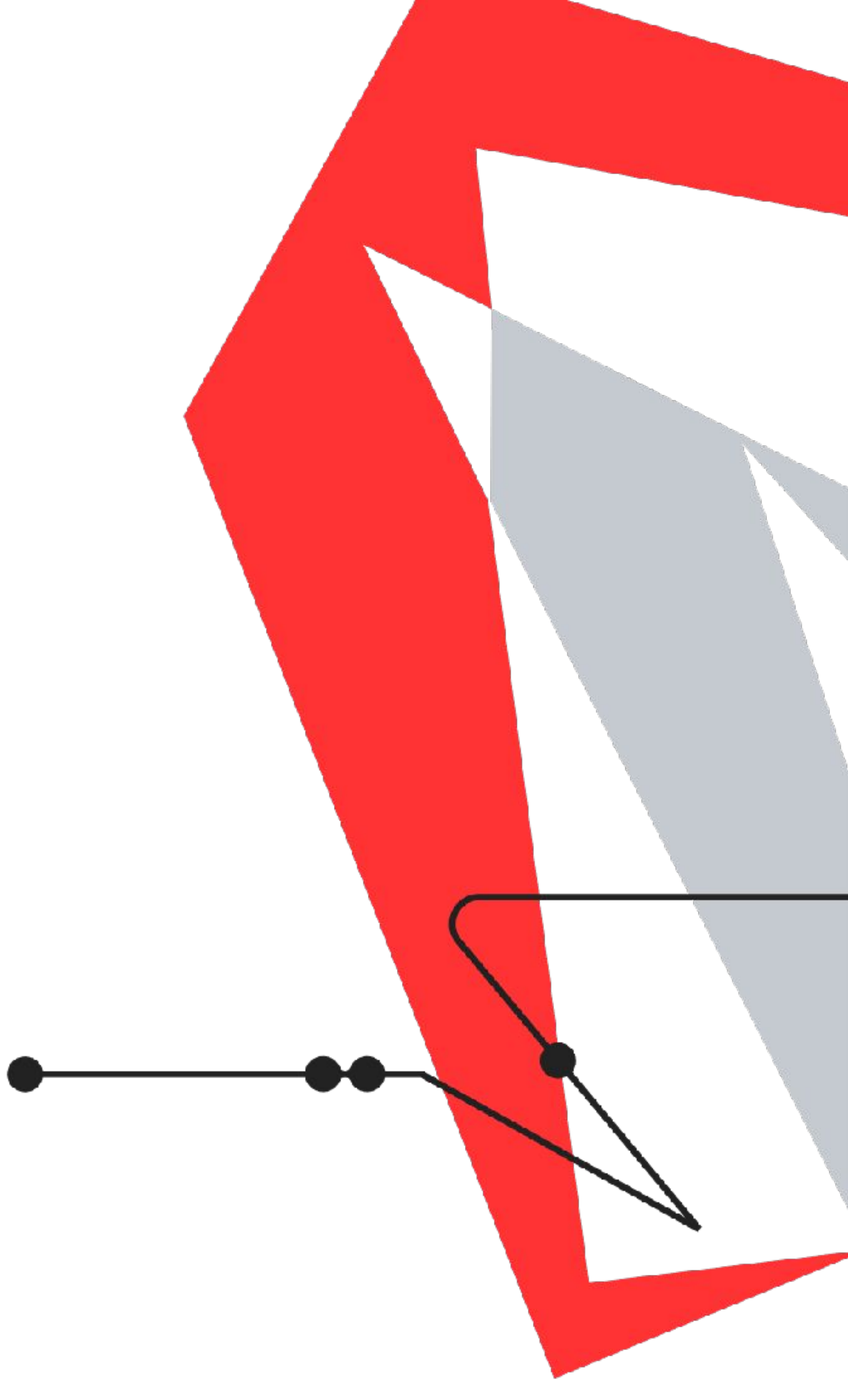
Code review rules
All passed
Details

Labels
Add label

Но если присмотреться, повсюду AI-native функциональность


1: OpenCode
SourceCraft
SourceCraft CodeAssistant CLI | powered by OpenCode
Выполни задачу из тикета #3
Build Default SourceCraft
tab agents ctrl+p commands
/tmp/coffeeshop:feature/dark-theme ⊙ 0 MCP /status
v1.4.6-sc

А что насчет
анализа
кода?



Diktat: Lightweight Static Analysis for Kotlin

Andrey Kuleshov[1], Petr Trifanov[1], Vladislav Frolov[1], Guangtai Liang[2]
[1] Russian Research Institute, Huawei Technologies Co., Ltd, Moscow, Russia
[2] Software Analysis Lab, Huawei Technologies Co., Ltd, Beijing, China
Email: {andrey.kuleshov, trifanov.petr, vladislav.frolov, liangguangtai}@huawei.com

Abstract—Kotlin is a relatively new programming language that is becoming more and more popular year after year. As the global codebase written in Kotlin grows, more use cases appear. To prevent code issues, developers require intelligent solutions for static analysis. These tools are built on top of different Intermediate Representations (IR), provided by the Kotlin compiler: Program Structure Interface (PSI), new Intermediate Representation (new IR), and Abstract Syntax Tree (AST). However, existing code analysis tools have limitations, including complexity, configurability, low performance, and the number of inspections. To address these limitations, we propose an approach of the lightweight static analysis for Kotlin, which was implemented in the Diktat project. We will also cover industrial problems of static analysis for Kotlin and will review the usage of different IRs for that purpose. Most popular open-source analyzers will be reviewed and compared by metrics of the resulting code.

Index Terms—Analysis, Reliability, Tools, Correctness proof

I. Introduction

Static code analysis is a method used to analyze the program and discover functional and stylistic issues without actually running it. Many developers, especially beginners and students, pay too little attention to the importance of writing clear and maintainable code [1]. This can lead to the fact that in mature projects developers can spend more time reading the code and synchronizing it with other developers than writing it. With certain research suggesting that the ratio of time spent reading versus writing can sometimes reach ten to one [2]. As developers spend most of their time reading and maintaining the code, it's important that they understand it properly. Static analysis helps different developers adhere to the same code style, guiding them to a better understanding of each other's intent and reducing the number of code errors. According to the recent studies, increasing number of developers are now using static analysis in their projects to prevent errors in the code [3].

However, as an area of Computer Science, static code analysis includes not only the implementation of analyzers for code error discovery, but also the writing of clear user-friendly warning messages, allowing for a better understanding and smoother workflow developers [4]. Different methods of static analysis can detect security vulnerabilities, architecture errors, problems with multi-threaded applications [5], and other issues. However, we can generally define four independent equivalence groups of problems that can be detected (and in some cases fixed) by static analysis: syntactic, semantic, functional, and security. Group of *functional* problems includes the detection of issues that can possibly break the business logic of an application (for example, possible Null Pointer Exceptions, Infinite Loops, Buffer Overflow Errors and so on). And *security* issues naturally covers security vulnerabilities. Meanwhile, the *syntactic* group includes features such as indentation, naming conventions, and comment formatting. While the *semantic* group includes architectural and design patterns, approaches for code modularization, the choice of names for methods and variables, and documentation styles. In most cases, however, it becomes very difficult or even impossible to implement the detection of deeply functional issues, as most of the modern open-source analyzers (including Diktat) are focused on the prevention of syntactic and semantic issues.

While it may seem that syntactic analysis - and even Semantic analysis - are straightforward processes, there remain many gaps in this well-studied area, such as new algorithms, False-positive (FP) ratio, and performance. Consequently, a lot of effort has been devoted to making static analysis faster [6]. In addition, the reduction of the False-positive rate is also becoming a research priority [7], because with a high rate of false-positives, developers start to ignore warnings, and it results to the degradation of code quality. That being said, static analysis is not only a scientific problem, but also an area of vital importance to the development of enterprise applications [8].

Kotlin, a new and modern programming language, faced many of the same issues related to static analysis that other mature languages have encountered a long time ago. However, the main difference is in the initial implementation of Kotlin compiler architecture [9]. Out-of-the-box, the Kotlin compiler provides the ability to parse the original code into Kotlin data structures that are expressed by different types of Intermediate Representation: AST, PSI and the new IR. We will focus more on these later in this paper, but for now it is important to understand that Kotlin is a general-purpose programming language built on top of the best ideas from modern JVM-languages [10]. Kotlin is currently rapidly growing in popularity [11]. It's language infrastructure includes mechanisms to operate with the IR of a code during a compilation process and making static analysis easier. Also there are several

Немного предистории: мой путь сквозь анализ кода

https://github.com/saveourtool/diktat

diktat Public

Edit Pins | Unwatch 8 | Fork 40 | Starred 572

master | 27 Branches | 47 Tags | Go to file T | Add file | Code

orchestr7 Support of Kotlin 2.1 and corresponding Ktlint 1.5 update (#1947) cd77a0b · 2 years ago 1,190 Commits

.git-hooks	Reused c.p.k.c.Rule.VisitorModifier.RunAfterRule (#1310)	4 years ago
.github	Support of Kotlin 2.1 and corresponding Ktlint 1.5 update (...	2 years ago
diktat-api	Support virtual path in DiktatProcessor (#1874)	3 years ago
diktat-cli	Supertypes should be on separate lines (#1902)	2 years ago
diktat-common-test	Fixed PARAMETER_NAME_IN_OUTER_LAMBDA (#1882)	3 years ago
diktat-dev-ksp	Fixed PARAMETER_NAME_IN_OUTER_LAMBDA (#1882)	3 years ago
diktat-gradle-plugin	Support of Kotlin 2.1 and corresponding Ktlint 1.5 update (...	2 years ago
diktat-ktlint-engine	Support of Kotlin 2.1 and corresponding Ktlint 1.5 update (...	2 years ago
diktat-maven-plugin	fix(diktat-maven-plugin): Maven projects with non local pa...	2 years ago
diktat-rules	Support of Kotlin 2.1 and corresponding Ktlint 1.5 update (...	2 years ago
diktat-ruleset	Support running diktat without configuration (#1837)	3 years ago
diktat-runner	Use diktat-runner:shadow as variant (#1875)	3 years ago
examples	Support of Kotlin 2.1 and corresponding Ktlint 1.5 update (...	2 years ago
gradle	Support of Kotlin 2.1 and corresponding Ktlint 1.5 update (...	2 years ago
info	chore(deps): update dependency gradle to v8.11.1 (#1945)	2 years ago
wp	Updating the package name to the new one (#1687)	3 years ago
.editorconfig	Updating the package name to the new one (#1687)	3 years ago
.gitattributes	DKT-13: [KTlint] support rule 1.3	6 years ago
.gitignore	Updated README.md for Diktat 2.0 (#1839)	3 years ago
CNAME	Create CNAME	2 years ago
CODE_OF_CONDUCT.md	Adding 3rd chapter of a codestyle (#65)	6 years ago

About

Strict coding standard for Kotlin and a custom set of rules for detecting code smells, code style issues and bugs

diktat.saveourtool.com

android | awesome | awesome-kotlin | checkstyle | code-checker | code-fixer | codecheck | codestyle | hacktoberfest | kotlin | ktlint | linter | static-analysis

Readme
MIT license
Code of conduct
Contributing
Activity
Custom properties
572 stars
8 watching
40 forks

Раньше мы думали про компоненты фронтенда компилятора, как исходный код будет обрабатываться и храниться

bare PSI No-Type-Resolution analysis. Imagine we have the following code:

```
if (a!=null) {}
```

This code will be represented with the following tree:

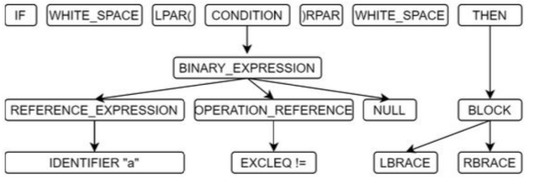


Figure 2. Example of the PSI tree

This code contains issues that can be detected using AST without type resolution. For example, missing WHITE_SPACEs in the BINARY_EXPRESSION or empty THEN block. These issues can also be automatically fixed by adding or removing proper AST nodes. However, there are issues in the code that are less trivial. For example, as Kotlin's type system is aimed at eliminating the danger of null references, the null check can be replaced with Kotlin-like statements:

```
a?.let {}
```

Such code can be automatically fixed without requiring type resolution (this rule is implemented in Diktat static analyzer). But what if the variable *a* is already *not-nullable*? If this case, there is no need to make a null check, and this code is redundant. This scenario cannot be covered by No-Type-Resolution analysis.

In its turn, the Kotlin compiler can also collect extra data and store it in a so-called *BindingContext* in case *classpath* was provided. This information contains all the required data relating to inferred types, type nullability, usages, calls, and so on. For example, using this information we can understand that the variable is nullable and null-checks are redundant.

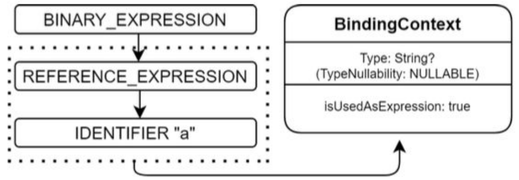


Figure 3. BindingContext is storing type information

The New IR introduced in Kotlin 1.4 also contains this information about usages and types. As such, the analysis done using BindingContext or IR will be called analysis With-Type-Resolution.

Such a technique requires resolving and loading dependencies, processing a large set of files, and preparing a full IR, which will then be further processed into platform-specific code. This is not applicable to cases where the analysis is done using a single file or a code snippet. In addition, as shown in the examples above it is possible to find and automatically fix a large number of lightweight problems with No-Type-Resolution analysis.

That being said, type resolution opens a path to more sophisticated analysis, but leads to the lower performance and to limitations related to context. IR looks promising in this regard, thanks to its enhanced and enriched tree-like representation. However, IR and compiler plugin API are still unstable and available only with Kotlin 1.4+. Also, we have shown in the Diktat static analyzer that the usage of certain heuristics enables complex analysis to be performed even without a type resolution.

IV. Existing Open-source Static Analyzers for Kotlin

Built-in Kotlin compiler's analyzer: Kotlin performs a handful of checks on its own during compilation time. These checks are emitted as warnings in the compiler output and can find shadowed names, unused variables, and so on. This tool is also reused in Intellij Idea IDE.

Ktlint[1]: A popular static analyzer for Kotlin code, Ktlint is aimed at retaining a unified style throughout a project. Under development by Pinterest since 2016, it can lint the code and fix discovered issues. Its checks operate on the top of AST/PSI, and Ktlint can be used as a standalone tool, or via third-party integration with Maven and Gradle. Ktlint has a small set of standard rules, which respect configuration options from .editorconfig. This analyzer supports suppression of rules via special custom comments.

Detekt[2]: A community-driven project, initiated in 2016. It operates over an abstract syntax tree (AST/PSI) and meta-information provided by the Kotlin compiler (BindingContext). On the top of that info, it performs a complex analysis of the code. However, this project is more focused on checking the code rather than fixing it (for automated fixers it reuses Ktlint as a plugin). It offers configurable rule sets and supports rules suppression on code fragments using *@Suppress* annotation. In addition, the number of checkers is large: with more than 100 inspections. Detekt offers first-party Gradle and IntelliJ integration, as well as third-party integrations for Maven, Bazel and Github actions.

Ktfmt[3]: A tool that formats Kotlin code (similar to google-java-format). Its development started at Facebook at the end of 2019. It can be added to a clients project

[1]https://github.com/pinterest/ktlint
[2]https://github.com/detekt/detekt
[3]https://github.com/facebookincubator/ktfmt

Выбирали лучшее Промежуточное Представление (IR) для анализа кода

AST?
PSI?
FIR? (Frontend IR)?
Enrichments?

Any other representation?

bare PSI No-Type-Resolution analysis. Imagine we have the following code:

```
if (a!=null) {}
```

This code will be represented with the following tree:

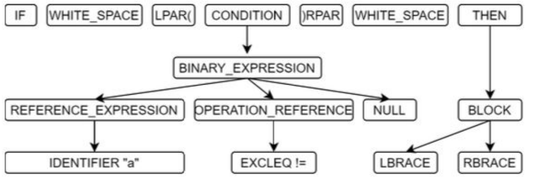


Figure 2. Example of the PSI tree

This code contains issues that can be detected using AST without type resolution. For example, missing WHITE_SPACEs in the BINARY_EXPRESSION or empty THEN block. These issues can also be automatically fixed by adding or removing proper AST nodes. However, there are issues in the code that are less trivial. For example, as Kotlin's type system is aimed at eliminating the danger of null references, the null check can be replaced with Kotlin-like statements:

```
a?.let {}
```

Such code can be automatically fixed without requiring type resolution (this rule is implemented in Diktat static analyzer). But what if the variable *a* is already *not-nullable*? If this case, there is no need to make a null check, and this code is redundant. This scenario cannot be covered by No-Type-Resolution analysis.

In its turn, the Kotlin compiler can also collect extra data and store it in a so-called *BindingContext* in case *classpath* was provided. This information contains all the required data relating to inferred types, type nullability, usages, calls, and so on. For example, using this information we can understand that the variable is nullable and null-checks are redundant.

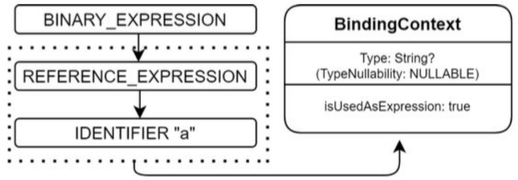


Figure 3. BindingContext is storing type information

The New IR introduced in Kotlin 1.4 also contains this information about usages and types. As such, the analysis done using BindingContext or IR will be called analysis With-Type-Resolution.

Such a technique requires resolving and loading dependencies, processing a large set of files, and preparing a full IR, which will then be further processed into platform-specific code. This is not applicable to cases where the analysis is done using a single file or a code snippet. In addition, as shown in the examples above it is possible to find and automatically fix a large number of lightweight problems with No-Type-Resolution analysis.

That being said, type resolution opens a path to more sophisticated analysis, but leads to the lower performance and to limitations related to context. IR looks promising in this regard, thanks to its enhanced and enriched tree-like representation. However, IR and compiler plugin API are still unstable and available only with Kotlin 1.4+. Also, we have shown in the Diktat static analyzer that the usage of certain heuristics enables complex analysis to be performed even without a type resolution.

IV. Existing Open-source Static Analyzers for Kotlin

Built-in Kotlin compiler's analyzer: Kotlin performs a handful of checks on its own during compilation time. These checks are emitted as warnings in the compiler output and can find shadowed names, unused variables, and so on. This tool is also reused in Intellij Idea IDE.

Ktlint[1]: A popular static analyzer for Kotlin code, Ktlint is aimed at retaining a unified style throughout a project. Under development by Pinterest since 2016, it can lint the code and fix discovered issues. Its checks operate on the top of AST/PSI, and Ktlint can be used as a standalone tool, or via third-party integration with Maven and Gradle. Ktlint has a small set of standard rules, which respect configuration options from .editorconfig. This analyzer supports suppression of rules via special custom comments.

Detekt[2]: A community-driven project, initiated in 2016. It operates over an abstract syntax tree (AST/PSI) and meta-information provided by the Kotlin compiler (BindingContext). On the top of that info, it performs a complex analysis of the code. However, this project is more focused on checking the code rather than fixing it (for automated fixers it reuses Ktlint as a plugin). It offers configurable rule sets and supports rules suppression on code fragments using *@Suppress* annotation. In addition, the number of checkers is large: with more than 100 inspections. Detekt offers first-party Gradle and IntelliJ integration, as well as third-party integrations for Maven, Bazel and Github actions.

Ktfmt[3]: A tool that formats Kotlin code (similar to google-java-format). Its development started at Facebook at the end of 2019. It can be added to a clients project

[1]https://github.com/pinterest/ktlint
[2]https://github.com/detekt/detekt
[3]https://github.com/facebookincubator/ktfmt

Некоторые языковые экосистемы даже глубоко интегрируют анализ кода непосредственно в компилятор или предоставляют возможность встраивать проверки в компилятор через плагины.

bare PSI No-Type-Resolution analysis. Imagine we have the following code:

```
if (a!=null) {}
```

This code will be represented with the following tree:

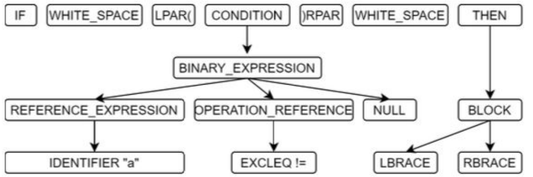


Figure 2. Example of the PSI tree

This code contains issues that can be detected using AST without type resolution. For example, missing WHITE_SPACEs in the BINARY_EXPRESSION or empty THEN block. These issues can also be automatically fixed by adding or removing proper AST nodes. However, there are issues in the code that are less trivial. For example, as Kotlin's type system is aimed at eliminating the danger of null references, the null check can be replaced with Kotlin-like statements:

```
a?.let {}
```

Such code can be automatically fixed without requiring type resolution (this rule is implemented in Diktat static analyzer). But what if the variable *a* is already *not-nullable*? If this case, there is no need to make a null check, and this code is redundant. This scenario cannot be covered by No-Type-Resolution analysis.

In its turn, the Kotlin compiler can also collect extra data and store it in a so-called *BindingContext* in case *classpath* was provided. This information contains all the required data relating to inferred types, type nullability, usages, calls, and so on. For example, using this information we can understand that the variable is nullable and null-checks are redundant.

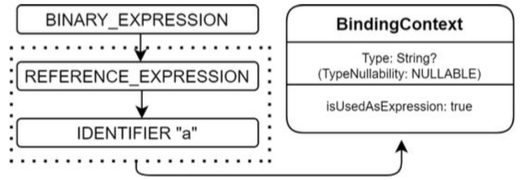


Figure 3. BindingContext is storing type information

The New IR introduced in Kotlin 1.4 also contains this information about usages and types. As such, the analysis done using BindingContext or IR will be called analysis With-Type-Resolution.

Such a technique requires resolving and loading dependencies, processing a large set of files, and preparing a full IR, which will then be further processed into platform-specific code. This is not applicable to cases where the analysis is done using a single file or a code snippet. In addition, as shown in the examples above it is possible to find and automatically fix a large number of lightweight problems with No-Type-Resolution analysis.

That being said, type resolution opens a path to more sophisticated analysis, but leads to the lower performance and to limitations related to context. IR looks promising in this regard, thanks to its enhanced and enriched tree-like representation. However, IR and compiler plugin API are still unstable and available only with Kotlin 1.4+. Also, we have shown in the Diktat static analyzer that the usage of certain heuristics enables complex analysis to be performed even without a type resolution.

IV. Existing Open-source Static Analyzers for Kotlin

Built-in Kotlin compiler's analyzer: Kotlin performs a handful of checks on its own during compilation time. These checks are emitted as warnings in the compiler output and can find shadowed names, unused variables, and so on. This tool is also reused in Intellij Idea IDE.

Ktlint[1]: A popular static analyzer for Kotlin code, Ktlint is aimed at retaining a unified style throughout a project. Under development by Pinterest since 2016, it can lint the code and fix discovered issues. Its checks operate on the top of AST/PSI, and Ktlint can be used as a standalone tool, or via third-party integration with Maven and Gradle. Ktlint has a small set of standard rules, which respect configuration options from .editorconfig. This analyzer supports suppression of rules via special custom comments.

Detekt[2]: A community-driven project, initiated in 2016. It operates over an abstract syntax tree (AST/PSI) and meta-information provided by the Kotlin compiler (BindingContext). On the top of that info, it performs a complex analysis of the code. However, this project is more focused on checking the code rather than fixing it (for automated fixers it reuses Ktlint as a plugin). It offers configurable rule sets and supports rules suppression on code fragments using *@Suppress* annotation. In addition, the number of checkers is large: with more than 100 inspections. Detekt offers first-party Gradle and IntelliJ integration, as well as third-party integrations for Maven, Bazel and Github actions.

Ktfmt[3]: A tool that formats Kotlin code (similar to google-java-format). Its development started at Facebook at the end of 2019. It can be added to a clients project

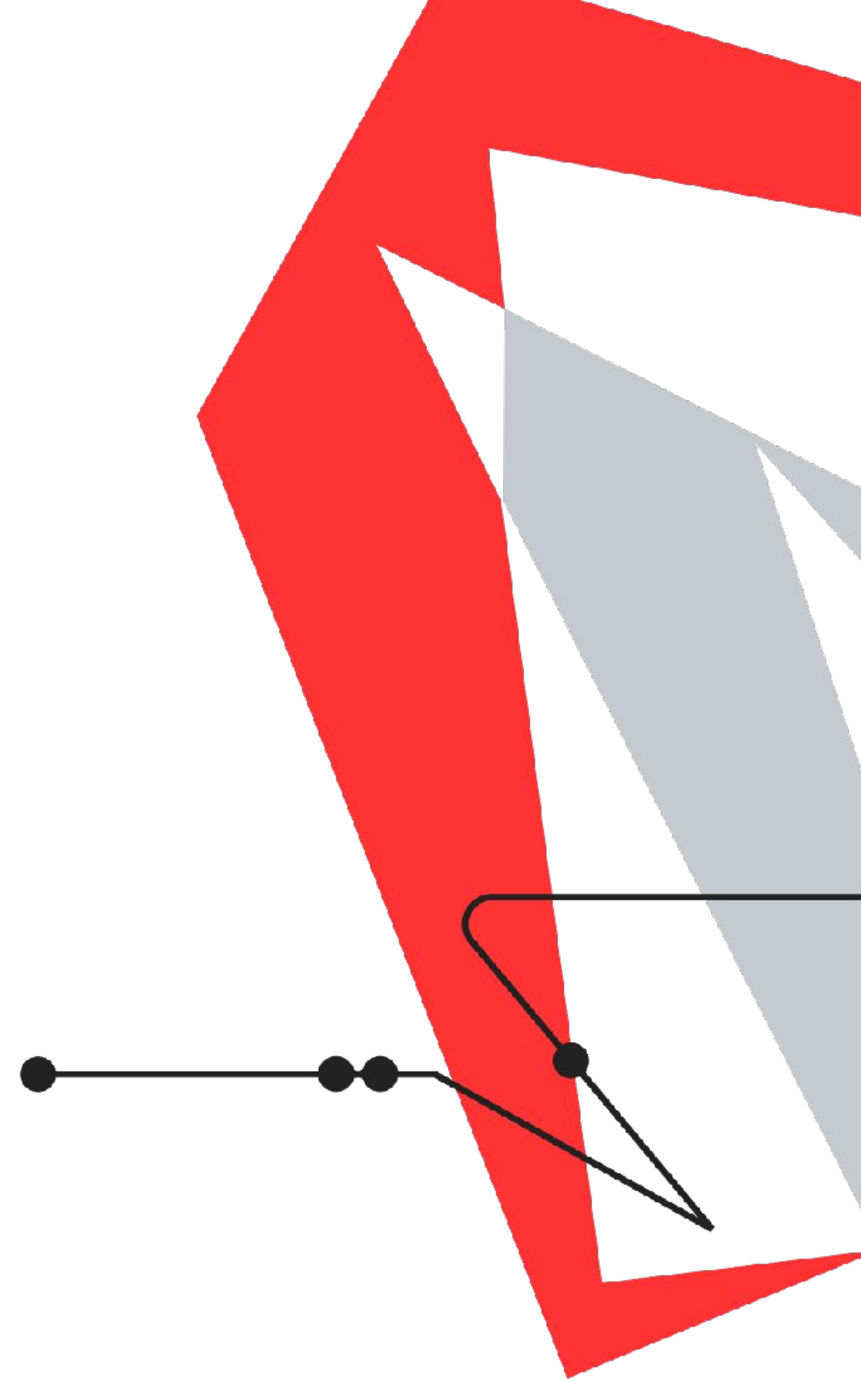
[1]https://github.com/pinterest/ktlint
[2]https://github.com/detekt/detekt
[3]https://github.com/facebookincubator/ktfmt

Три всадника AppSec Апокалипсиса



OWASP 2025

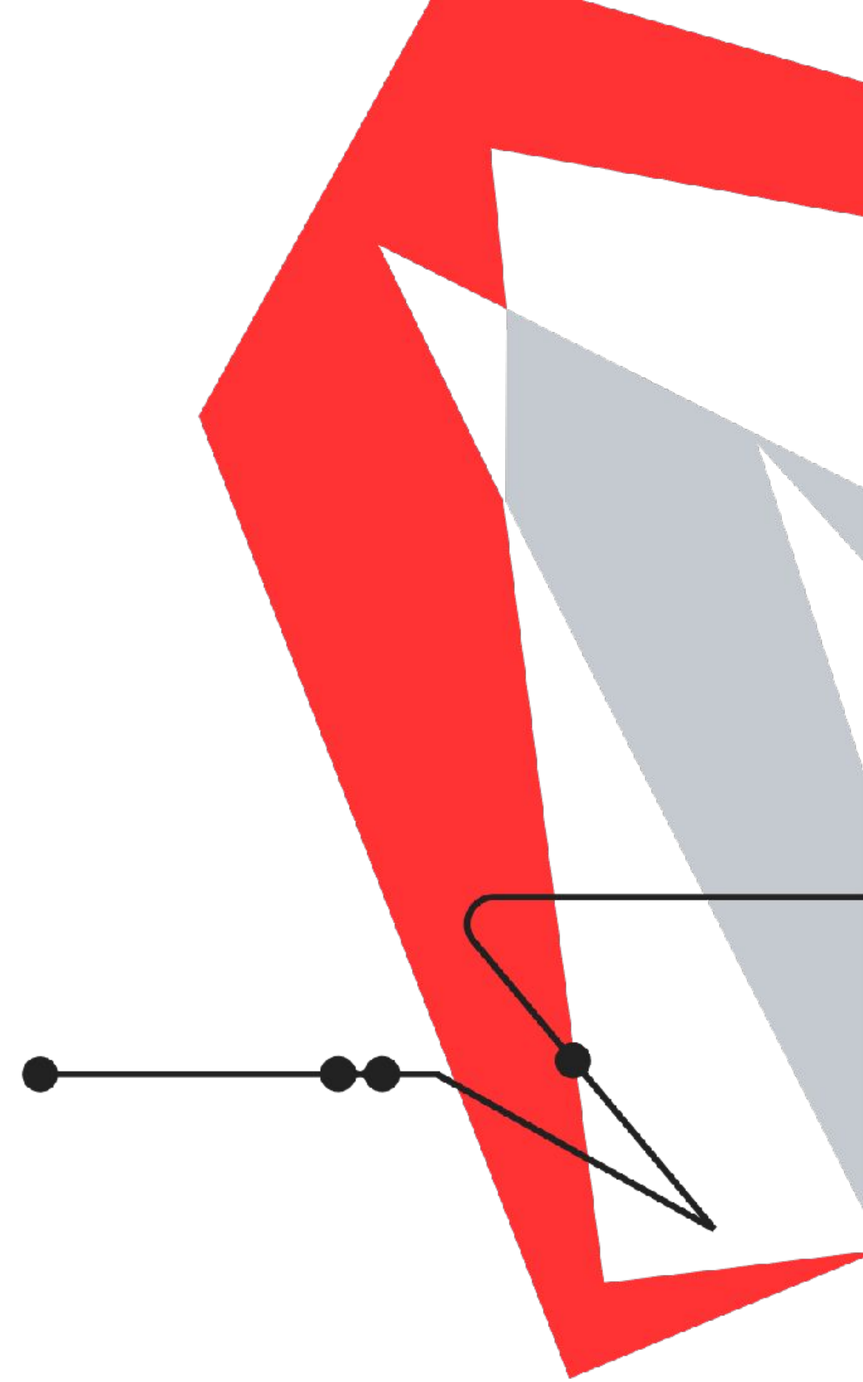
A01 Broken Access Control
A02 Security Misconfiguration
A03 Software Supply Chain Failures
A04 Cryptographic Failures
A05 Injection
A06 Insecure Design
A07 Authentication Failures
A08 Software or Data Integrity Failures
A09 Logging and Alerting Failures
A10 Mishandling of Exceptional Conditions



OWASP 2025

Остается с 2021

A01 Broken Access Control
A02 Security Misconfiguration
A03 Software Supply Chain Failures
A04 Cryptographic Failures
A05 Injection
A06 Insecure Design
A07 Authentication Failures
A08 Software or Data Integrity Failures
A09 Logging and Alerting Failures
A10 Mishandling of Exceptional Conditions

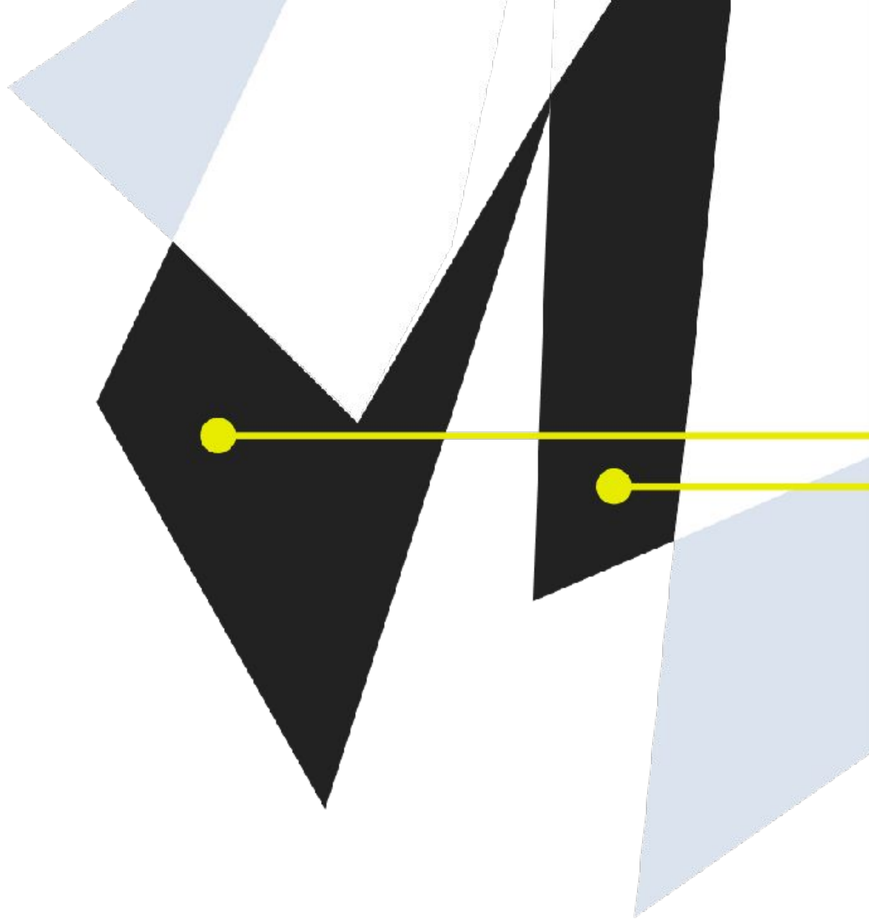


Можно решить алгоритмическим подходом и старыми добрыми классическими инструментами.

A02 Security Misconfiguration
A03 Software Supply Chain Failures

A04 Cryptographic Failures
A05 Injection

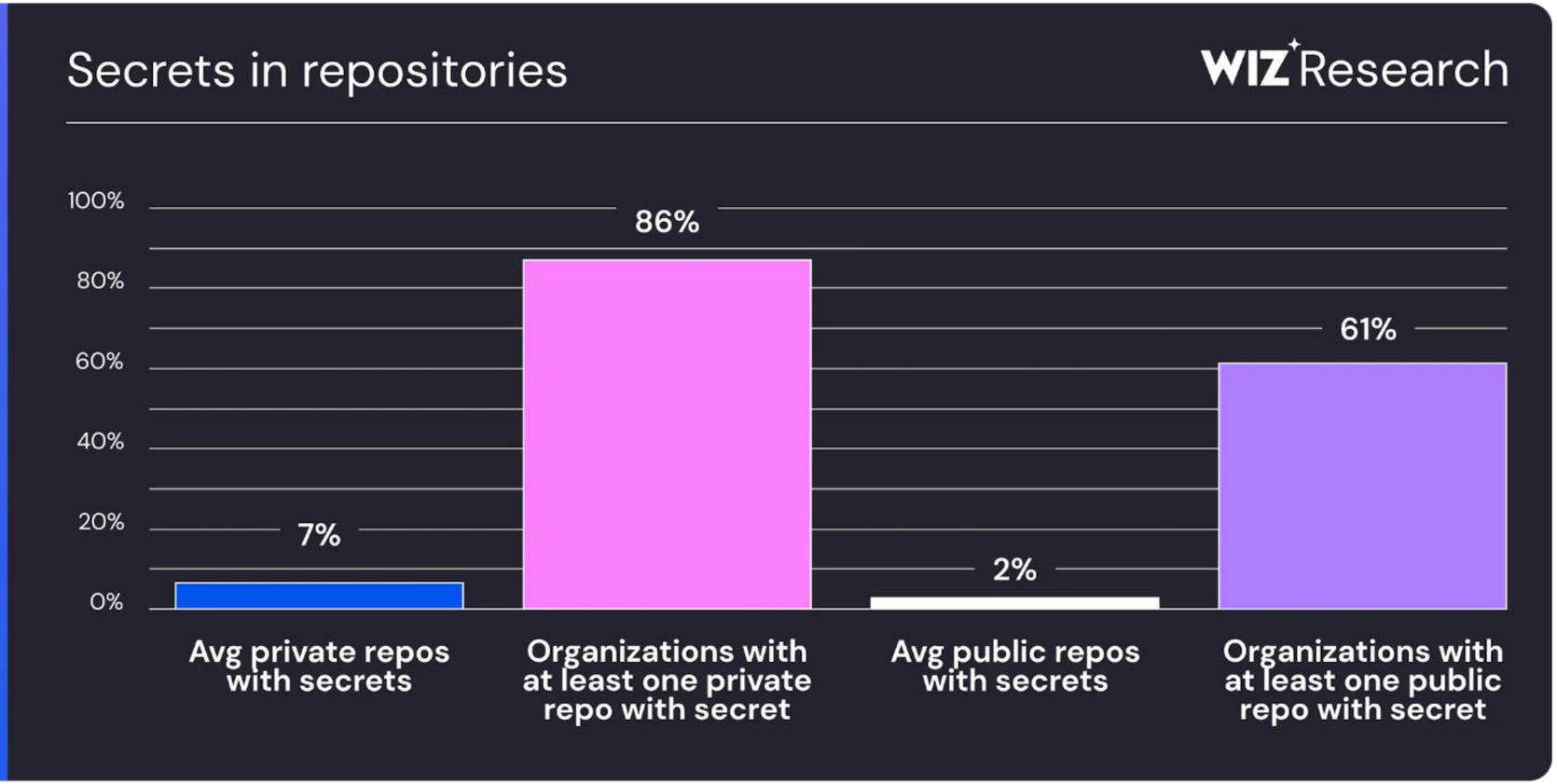
A10 Mishandling of Exceptional Conditions



Great example: Secrets

A04 Cryptographic Failures

Against the best practices, private repositories are used as a place to store secrets



Так просто!

```
[[rules]]
id = "yandex-access-token"
description = "Found a Yandex Access Token,
posing a risk to Yandex service
integrations and user data privacy."
regex = '''(?i)[\w.-]{0,50}?(?:yandex)(?:[
\t\w.-]{0,20})[\s'"]{0,3}(?:=▷|:{1,3}=|\|\
||:⟹|\?=|,)[\x60'"\s=]{0,5}(t1\.[A-Z0-9a-
z_-]+[=]{0,2}\.[A-Z0-9a-z_-]{86}[=]{0,2})(?
:[\x60'"\s;]|\\[nr]|$)'''
keywords = ["yandex"]
```

Shannon entropy + **Эвристики**
+ Leak Services

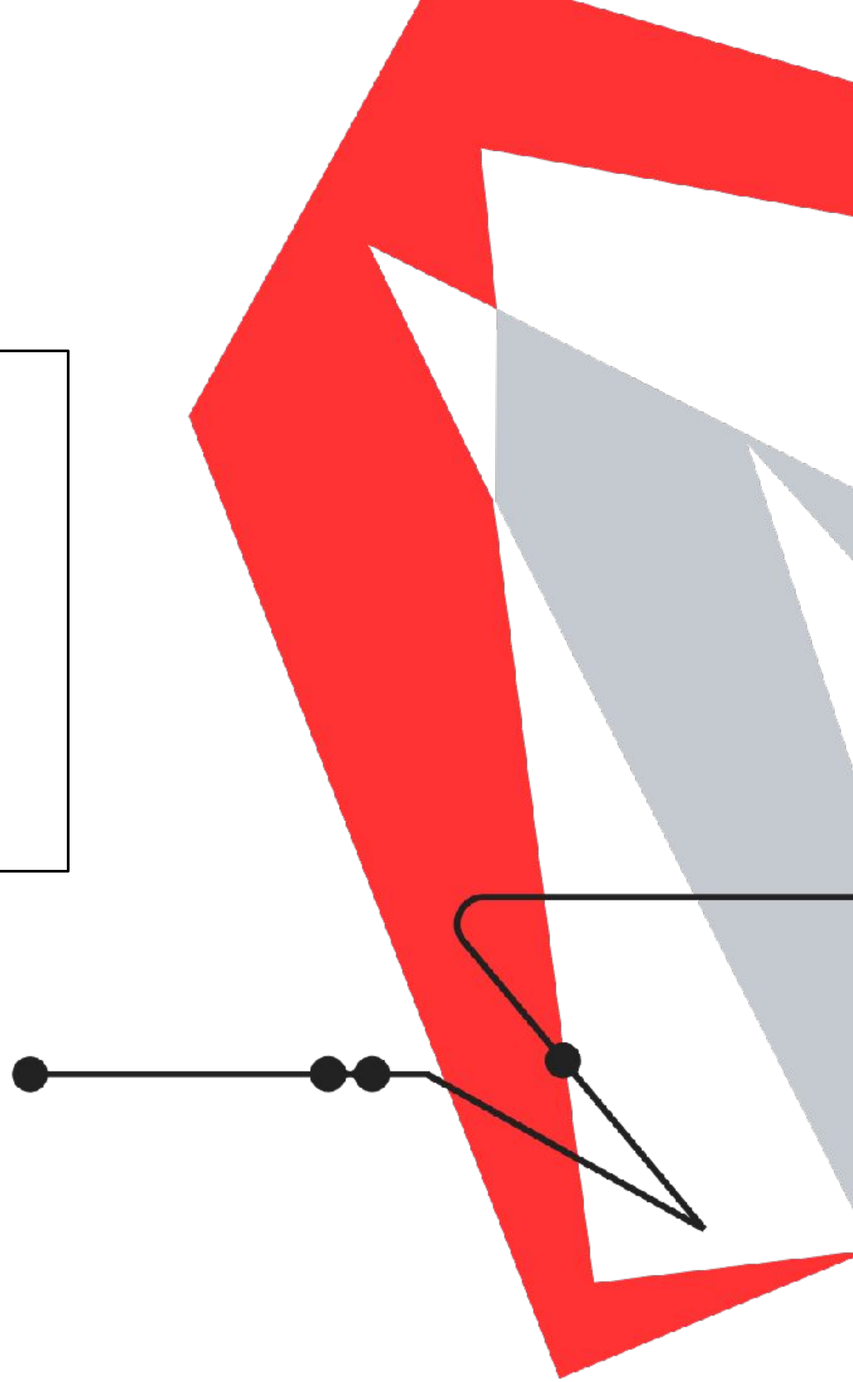
AKIAIOSFODNN7EXAMPLE – похоже на ключ
AAAAAAAAAAAAAAAA – очевидно мусор

$$H(X) = -\sum_i p_i \log_2 p_i$$

Gitleaks, trufflehog, kingfisher, etc.

Принцип один и тот же

Правило для анализатора ≈ Некоторая эвристика



False Positive

The analyzer reports a problem, but the code is actually safe.



Static Analyzer

Potential issue detected

Rule:	Possible null dereference
Location:	example.c:9
Symbol:	printf
Severity:	Medium

Actual Result

Safe

The code is correct.

- ✓ Condition ensures x > 0
- ✓ No null dereference
- ✓ No issue in this path

A false positive occurs when the static analyzer reports a problem that, after analysis, is determined to be harmless.

Analyzer flags an issue → Developer reviews → Code is safe

False Positive

The analyzer reports a problem, but the code is actually safe.

И единственный способ их РАЗОБРАТЬ / ПРОТРИАЖИТЬ — проверять их вручную…

example.c

```c
#include <stdio.h>

int max(int a, int b) {
    return (a > b) ? a : b;
}

void print_positive(int x) {
    if (x > 0) {
        printf("%d is positive\n", x);
    }
}

// All good here.
```

Code is correct ✓

Rule:	Possible null dereference
Location:	example.c:9
Symbol:	printf
Severity:	Medium

Actual Result

Safe

The code is correct.

- ✓ Condition ensures x > 0
- ✓ No null dereference
- ✓ No issue in this path

A false positive occurs when the static analyzer reports a problem that, after analysis, is determined to be harmless.

Analyzer flags an issue → Developer reviews → Code is safe

False Positive

The analyzer reports a problem, but the code is actually safe.

AI

example.c

```c
#include <stdio.h>

int max(int a, int b) {
    return (a > b) ? a : b;
}

void print_positive(int x) {
    if (x > 0) {
        printf("%d is positive\n", x);
    }
}

// All good here.
```

Code is correct ✓

Rule: Possible null dereference
Location: example.c:9
Symbol: printf
Severity: Medium

Actual Result

Safe

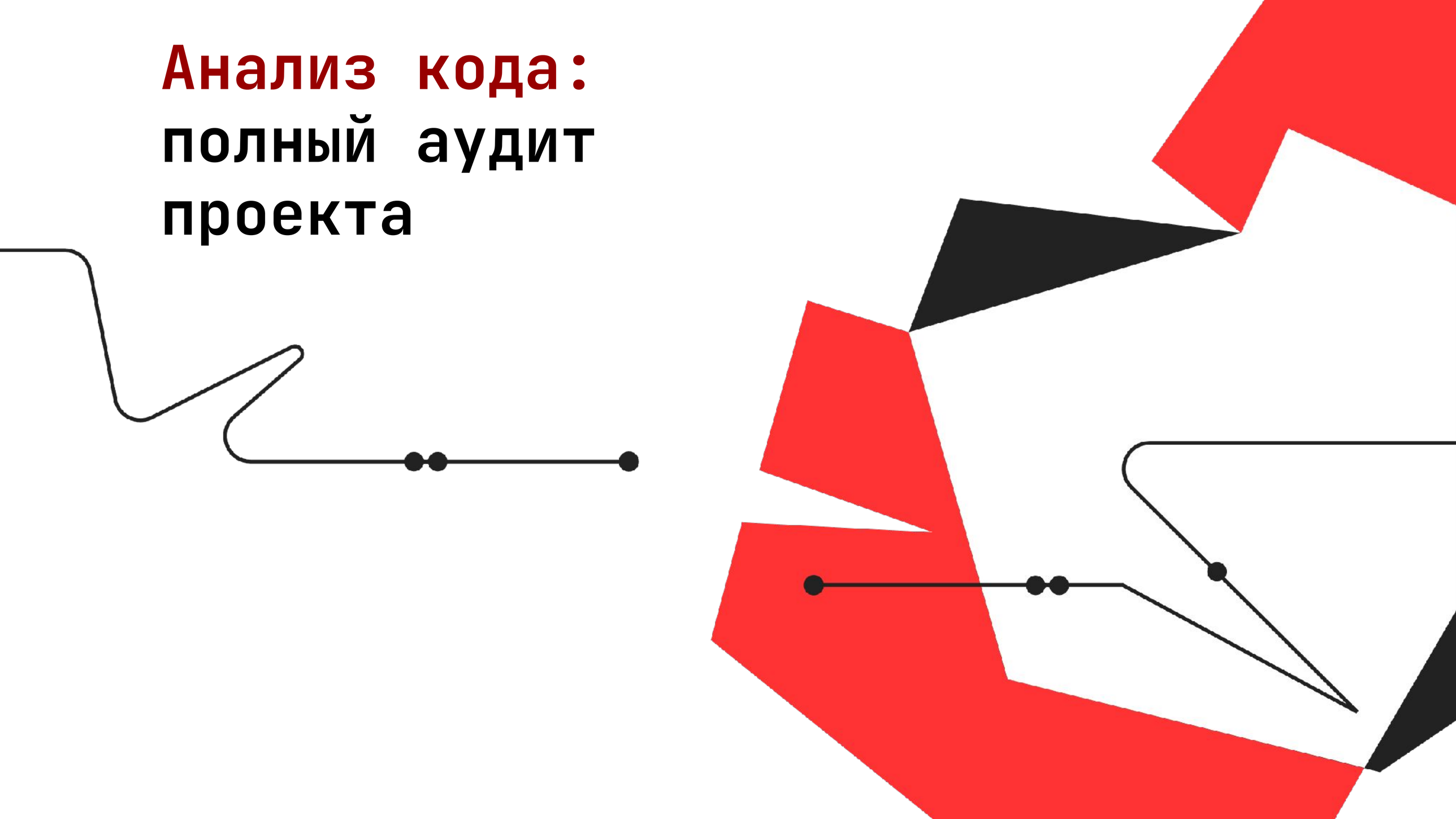
The code is correct.

- ✓ Condition ensures x > 0
- ✓ No null dereference
- ✓ No issue in this path

A false positive occurs when the static analyzer reports a problem that, after analysis, is determined to be harmless.

Analyzer flags an issue → Developer reviews → Code is safe

Анализ кода: полный аудит проекта





AI

Возвращаясь к OWASP

A01 Broken Access Control

Нужно понимать бизнес-логику

Реальный пример из моего кода



AI

SAST report — main-testing (local)
Found (verified): High 8, Medium 14, Low 8

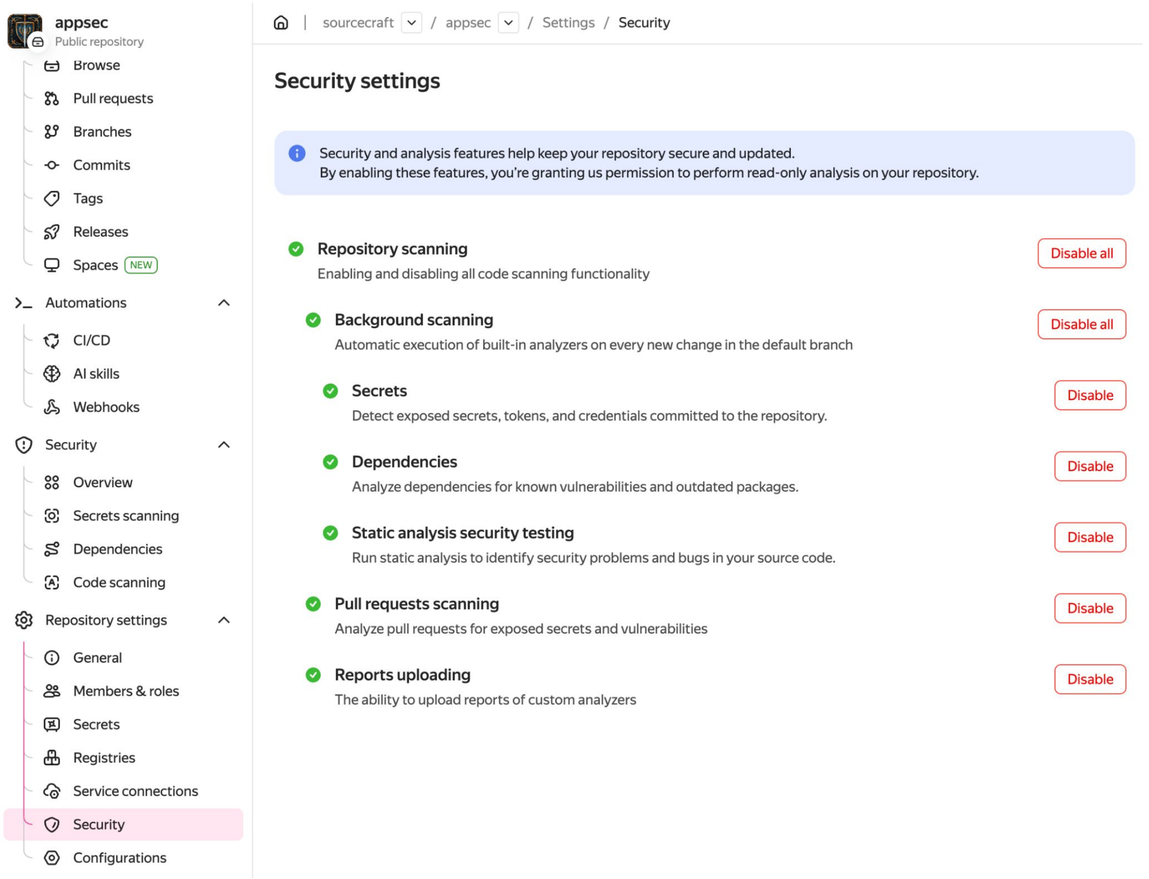
What matters (TL;DR)
- **High** · **broken-access-control** · `appsec-backend/src/main/kotlin/dev/sourcecraft/api/grpc/read/AdminCommonServiceGrpcApi.kt:139` — **A read-only viewer** (GET_APPSEC_RESULTS) of a repository can disable AppSec/PR-scan scanning for that repository, alter engine-type statuses, and **can force scan triage.**

Реальный пример из моего кода



AI

```
override fun readAppsecSettings(
  ...
) = controllersGrpcAPI.readApi() {
```



Реальный пример из моего кода

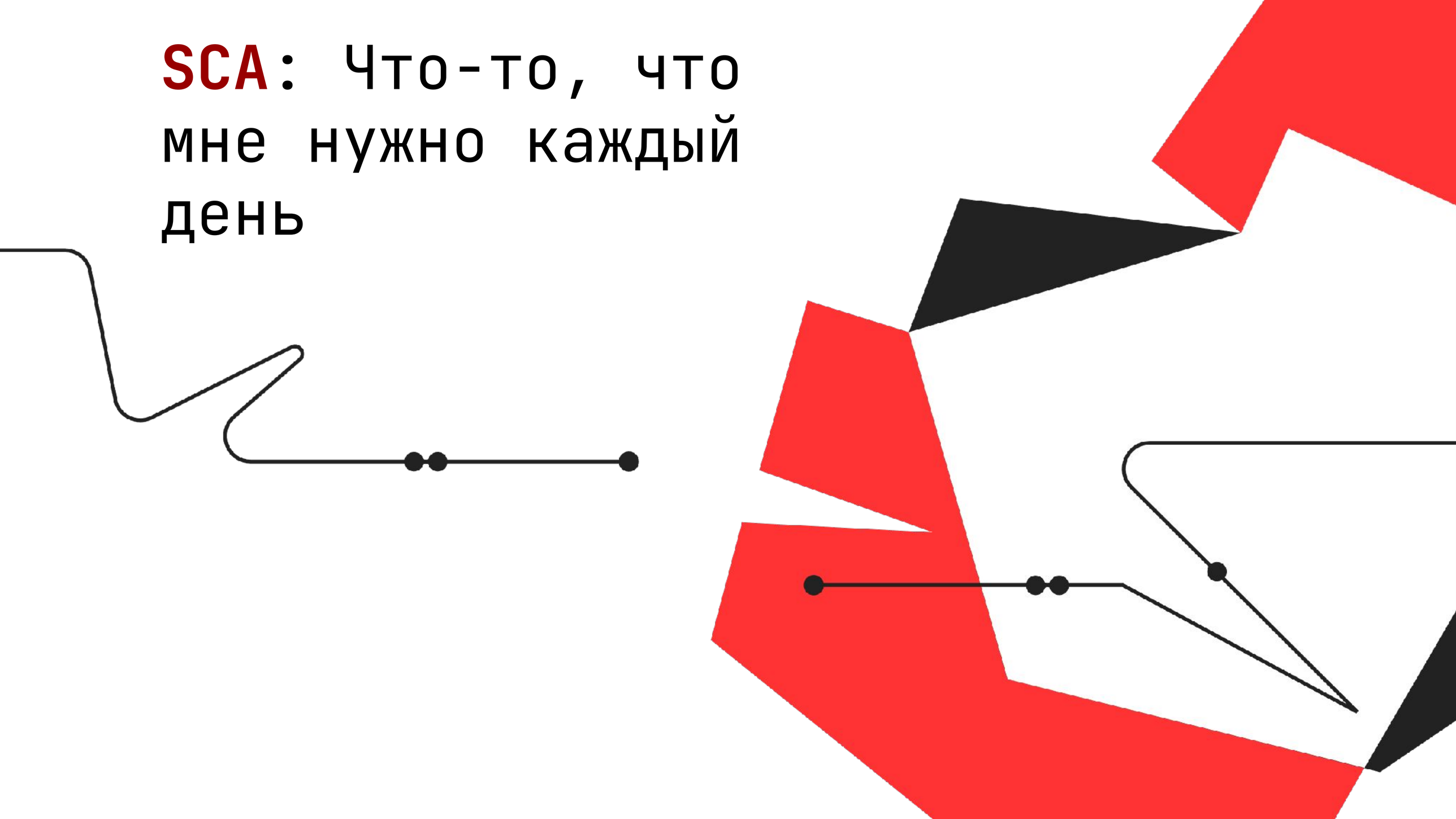


AI

```
override fun readAppsecSettings(
  ...
) = controllersGrpcAPI.readApi() {

      ...
    // Trigger the scan
      scanTriggerService.processSimpleAppsecTrigger(
         triggerRequest,
         apiContext
      )
}
```

SCA: Что-то, что мне нужно каждый день



Netty is an asynchronous, event-driven network application framework. ...

Last detected: 18 Jun 2026 • Trivy4Cdx • netty-codec-http@4.1.132.Final

CVE-2026-42587

Netty is an asynchronous, event-driven network application framework. Prior to 4.2.13.Final and 4.1.133.Final, HttpContentDecompressor accepts a maxAllocation parameter to limit decompression buffer size and prevent decompression bomb attacks. This limit is correctly enforced for gzip and deflate encodings via ZlibDecoder, but is silently ignored when the content encoding is br (Brotli), zstd, or snappy. An attacker can bypass the configured decompression limit by sending a compressed payload with Content-Encoding: br instead of Content-Encoding: gzip, causing unbounded memory allocation and out-of-memory denial of service. The same vulnerability exists in DelegatingDecompressorFrameListener for HTTP/2 connections. This vulnerability is fixed in 4.2.13.Final and 4.1.133.Final.

Links 2

https://avd.aquasec.com/nvd/cve-2026-42587

https://nvd.nist.gov/vuln/detail/cve-2026-42587

Activity

Add your markup here

Comment

04 Jun 2026

SourceCraft Security Bot has opened this alert first time • 04 Jun 2026, 11:10

Alert details

CVE-2026-42587

CVSS: 7.5 (High)

Last detected: 18 Jun 2026

Repository

sourcecraft/appsec

release/0.3.x

2c67f60

Package

io.netty:netty-codec-http2

4.2.13.Final, 4.1.133.Final

Patched version

```
CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H
```

appsec
Public repository

Search

Plan
- Issues
- Milestones
- Labels

Code
- Browse
- Pull requests
- Branches
- Commits
- Tags
- Releases
- Spaces NEW

Automations
- CI/CD
- AI skills
- Webhooks

Security
- Overview
- Secrets scanning
- Dependencies
- Code scanning

Repository settings

Netty is an asynchronous, event-driven network application framework. ...

Last detected: 18 Jun 2026 • Trivy4Cdx • netty-codec-http@4.1.132.Final

CVE-2026-42587

Netty is an asynchronous, event-driven network application framework. Prior to 4.2.13.Final and 4.1.133.Final, HttpContentDecompressor accepts a maxAllocation parameter to limit decompression buffer size and prevent decompression bomb attacks. This limit is correctly enforced for gzip and deflate encodings via ZlibDecoder, but is silently ignored when the content encoding is br (Brotli), zstd, or snappy. An attacker can bypass the configured decompression limit by sending a compressed payload with Content-Encoding: br instead of Content-Encoding: gzip, causing unbounded memory allocation and out-of-memory denial of service. The same vulnerability exists in DelegatingDecompressorFrameListener for HTTP/2 connections. This vulnerability is fixed in 4.2.13.Final and 4.1.133.Final.

Links 2

https://avd.aquasec.com/nvd/cve-2026-42587

https://nvd.nist.gov/vuln/detail/cve-2026-42587

Что с фиксом?

Activity

Add your markup here

Comment

04 Jun 2026

SourceCraft Security Bot has opened this alert first time • 04 Jun 2026, 11:10

Alert details

CVE-2026-42587

CVSS: 7.5 (High)

Last detected: 18 Jun 2026

Repository

sourcecraft/appsec

release/0.3.x

2c67f60

Package

io.netty:netty-codec-http2

4.2.13.Final, 4.1.133.Final

Patched version

```
CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H
```

Update dependency org.junit.jupiter:junit-jupiter-params to v6 #393

Checks failing | Code

Open | renovate[bot] wants to merge 1 commit into `main` from `renovate/org.junit.jupiter-junit-jupiter-params-6.x`

Conversation 0 | Commits 1 | Checks 6 | Files changed 1 | +1 -1

renovate Bot commented on Jul 6 • edited — Contributor

This PR contains the following updates:

Package	Change	Age	Adoption	Passing	Confidence
org.junit.jupiter:junit-jupiter-params (source)	`5.14.1` → `6.1.3`	15d	8%	65%	low

Release Notes

▶ junit-team/junit-framework (org.junit.jupiter:junit-jupiter-params)

Configuration

📅 **Schedule**: (UTC)

- Branch creation
  - "before 4am on Monday"
- Automerge
  - At any time (no schedule defined)

🚦 **Automerge**: Disabled by config. Please merge this manually once you are satisfied.

♻ **Rebasing**: Whenever PR is behind base branch, or you tick the rebase/retry checkbox.

🔕 **Ignore**: Close this PR and you won't be reminded about this update again.

Reviewers

No reviews—at least 1 approving review is required.

Still in progress? Convert to draft

Assignees

No one—assign yourself

Labels

None yet

Projects

None yet

Milestone

No milestone

Development

Successfully merging this pull request may close these issues.

None yet

Notifications Customize

Unsubscribe

You're receiving notifications because you're watching this repository.

Update dependency org.junit.jupiter:junit-jupiter-params to v6 #393

Checks failing | Code

Open | renovate[bot] wants to merge 1 commit into `main` from `renovate/org.junit.jupiter-junit-jupiter-params-6.x`

Conversation 0 | Commits 1 | Checks 6 | Files changed 1 | +1 -1

renovate Bot commented on Jul 6 • edited — Contributor

This PR contains the following updates:

Package	Change	Age	Adoption	Passing	Confidence
org.junit.jupiter:junit-jupiter-params (source)	5.14. 6.1.3				



Release Notes

▶ junit-team/junit-framework (org.junit.jupiter:junit-jup

Configuration

📅 **Schedule**: (UTC)

- Branch creation
  - "before 4am on Monday"
- Automerge
  - At any time (no schedule defined)

🚦 **Automerge**: Disabled by config. Please merge this manually once you are satisfied.

♻ **Rebasing**: Whenever PR is behind base branch, or you tick the rebase/retry checkbox.

🔕 **Ignore**: Close this PR and you won't be reminded about this update again.

Reviewers
No reviews—at least 1 approving review is required.
Still in progress? Convert to draft

Assignees
No one—assign yourself

Labels
None yet

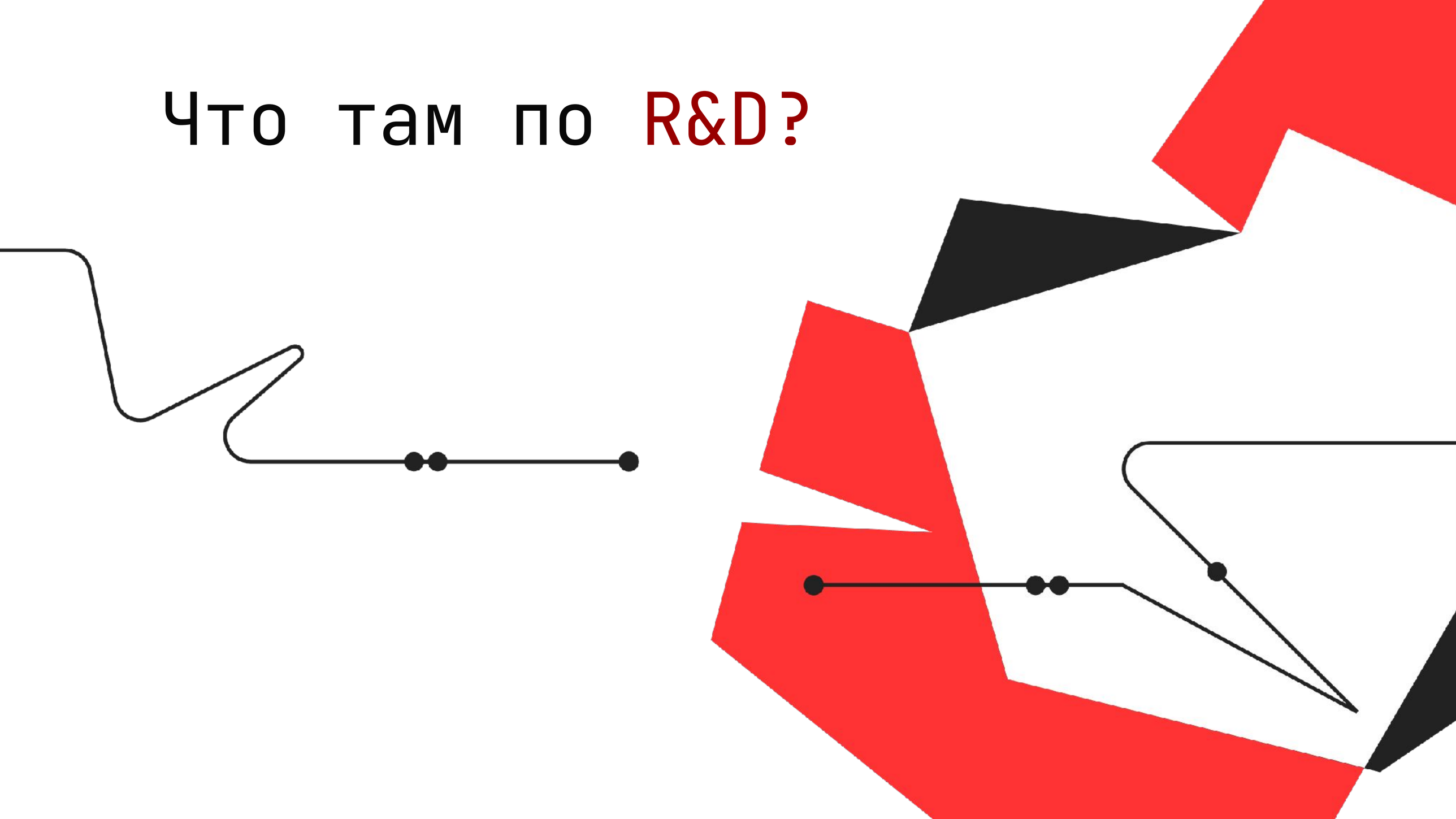
Projects
None yet

Milestone
No milestone

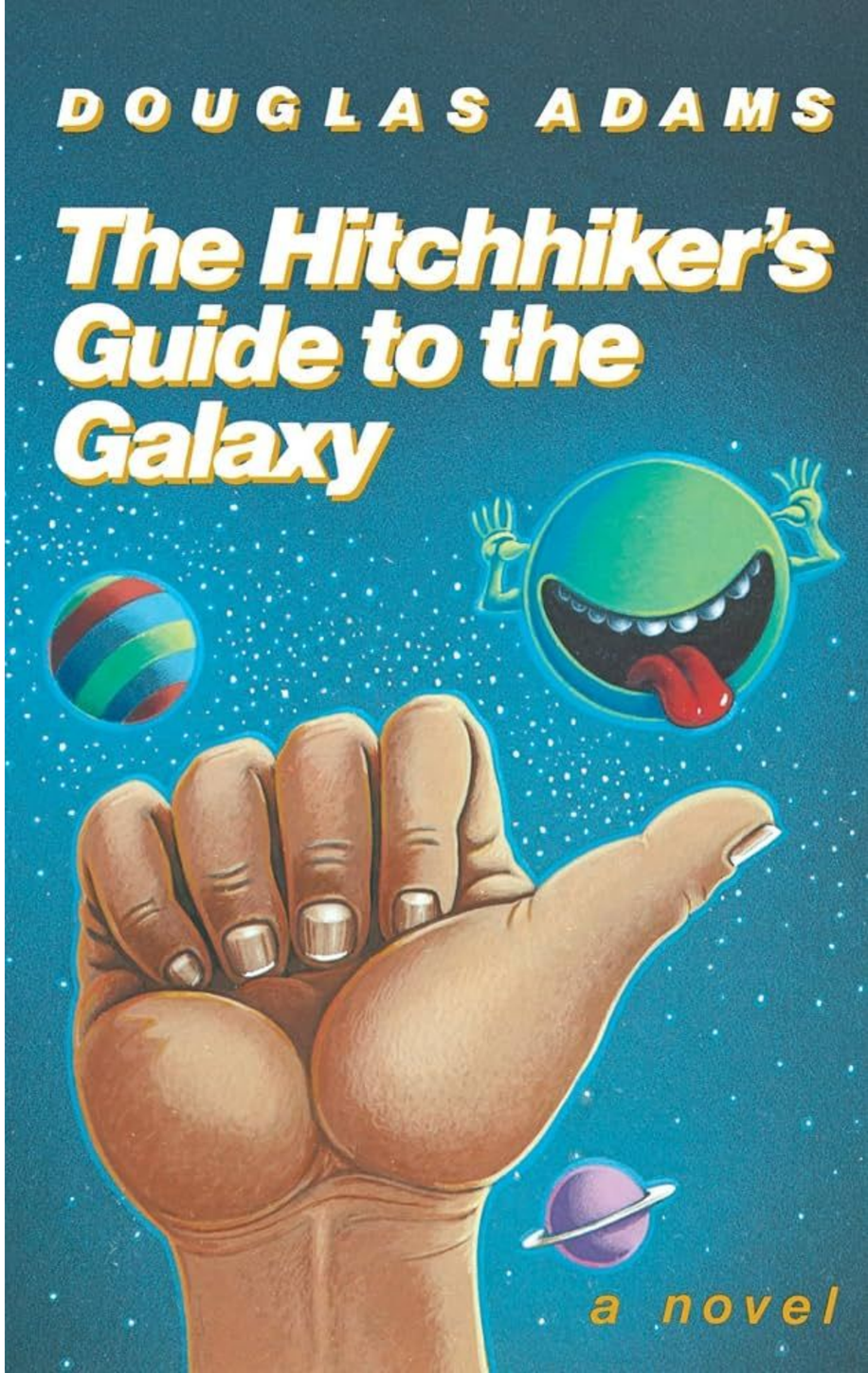
Development
Successfully merging this pull request may close these issues.
None yet

Notifications Customize
Unsubscribe
You're receiving notifications because you're watching this repository.

Что там по R&D?



The Hitchhiker's Guide to Program Analysis



The Hitchhiker's Guide to Program Analysis

2023 — LLift / **Part I: *A Journey with LLMs***



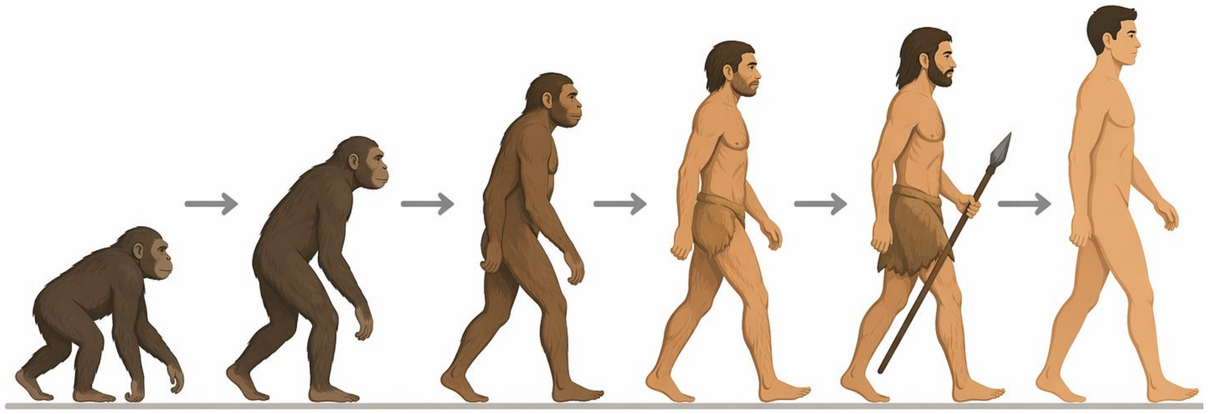
2025 — BugLens / **Part II: *Deep Thoughts by LLMs***



2026 — **Part III: Mostly Harmless LLMs**

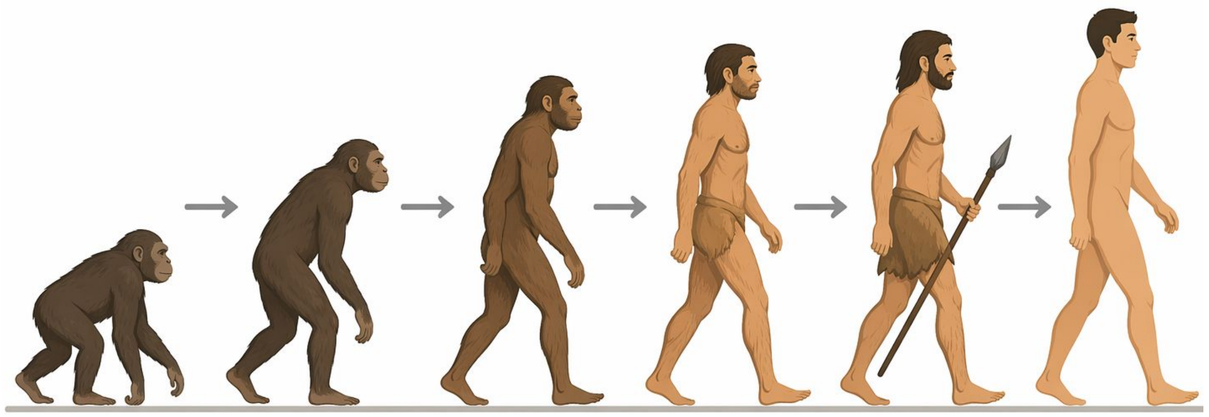


Мне нравится эта эволюция!



2023: LLM assists static analysis (particular rule, FP/non-FP?)

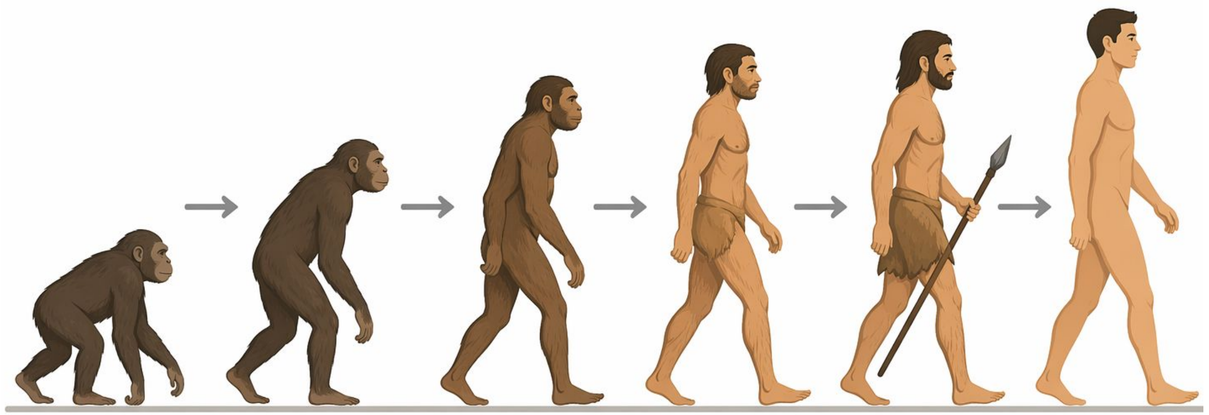
Мне нравится
эта эволюция!



2023: LLM assists static analysis (particular rule, FP/non-FP?)

2025: LLM performs structured program-analysis reasoning (reachability, criticality, constraints)

Мне нравится эта эволюция!

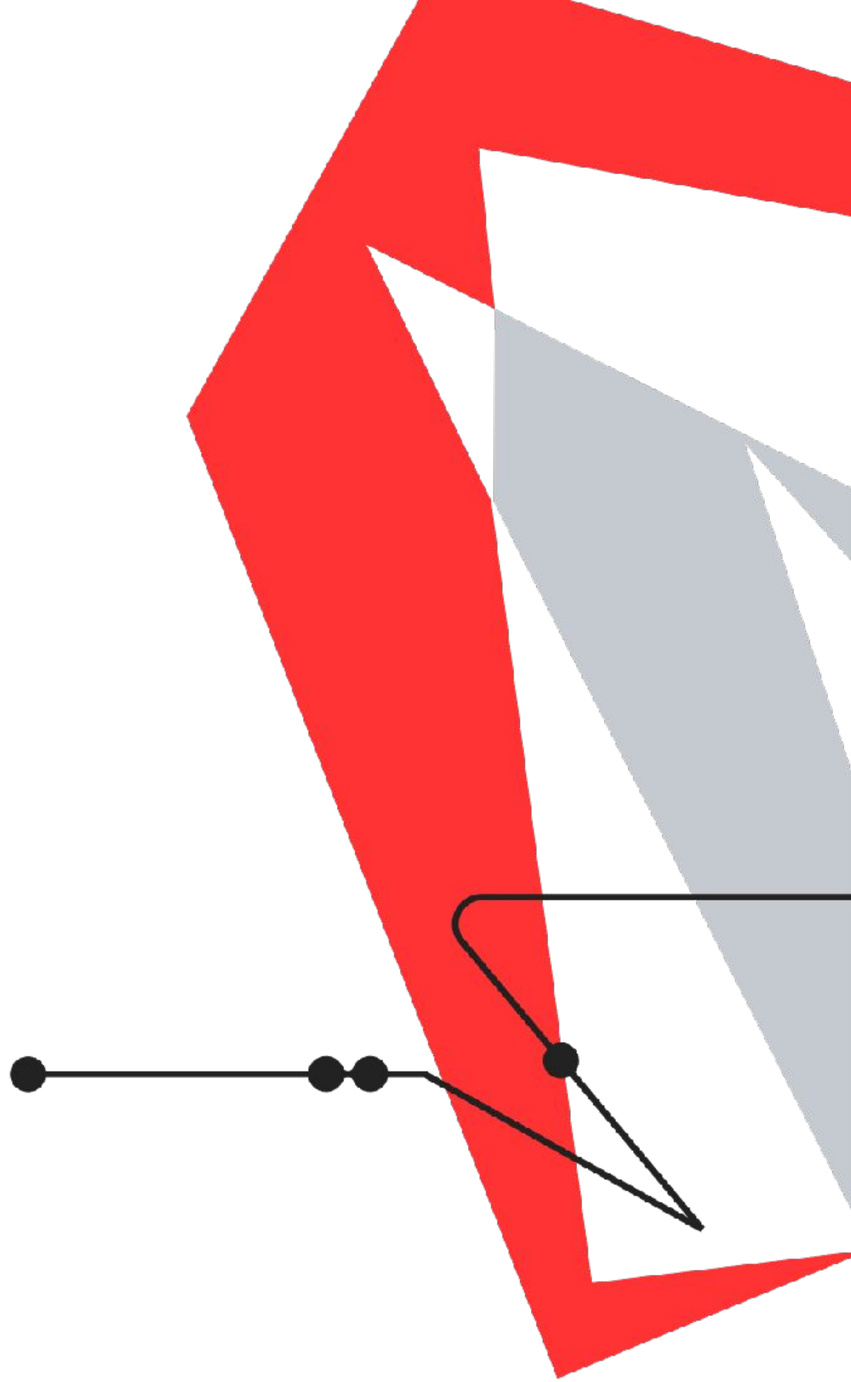


2023: LLM assists static analysis (particular rule, FP/non-FP?)

2025: LLM performs structured program-analysis reasoning (reachability, criticality, constraints)

2026: LLM orchestrates formal analysis instead **of being trusted as the oracle** (LLMs construct analysis; formal tools verify)

Двухфазный поиск уязвимостей от CloudFlare



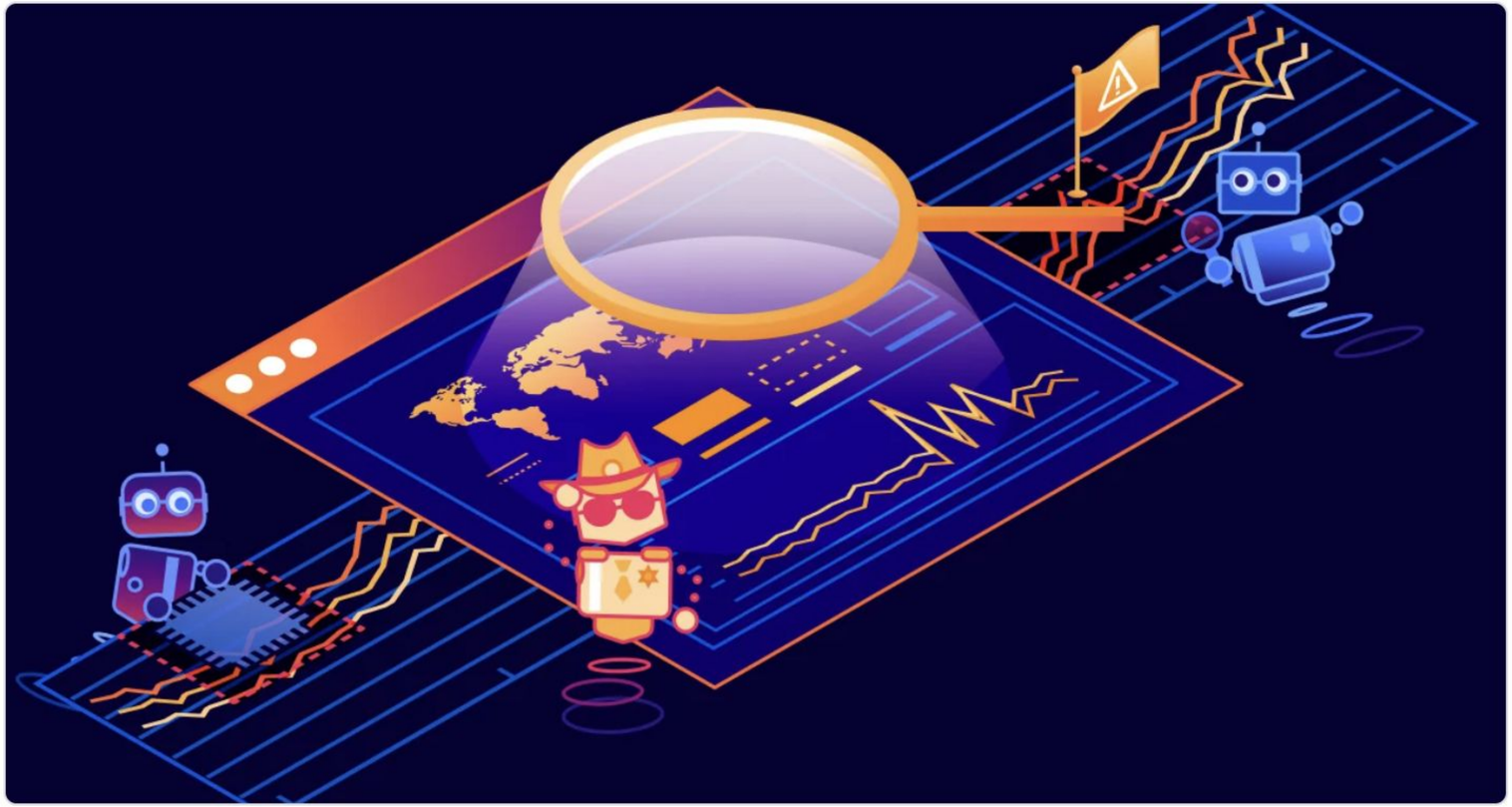
JUNE 18, 2026

Build your own vulnerability harness



Dan Jones, Alexandra Godoi, and Grant Bourzikas

21 MINUTE READ COPY URL



https://blog.cloudflare.com/build-your-own-vulnerability-harness/

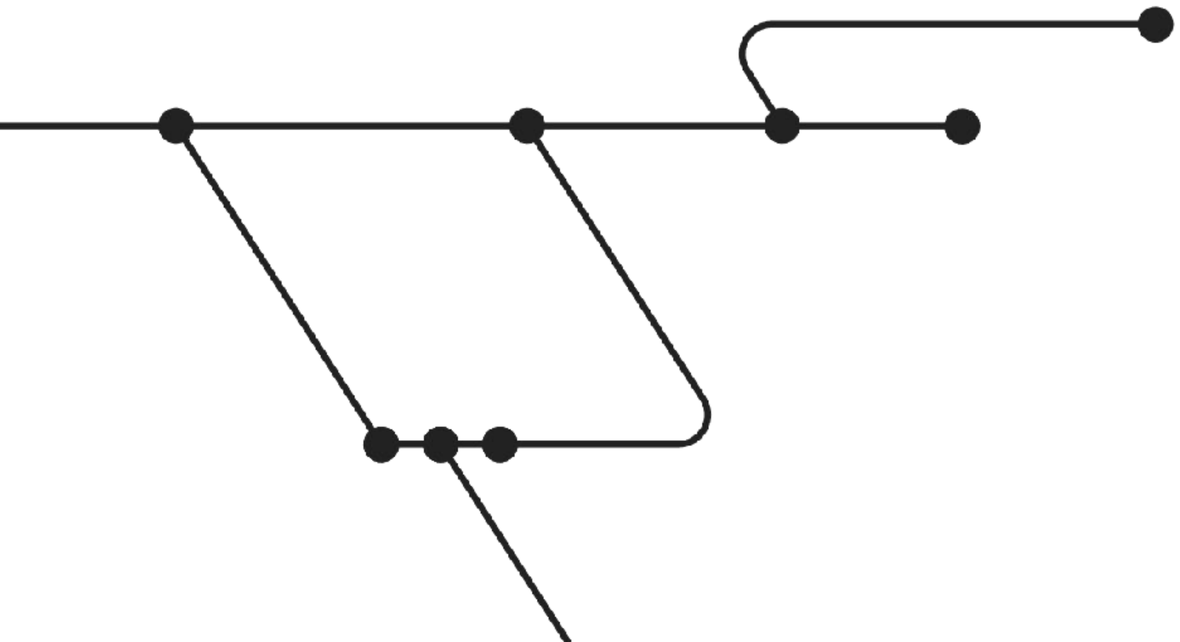
Подход простой!

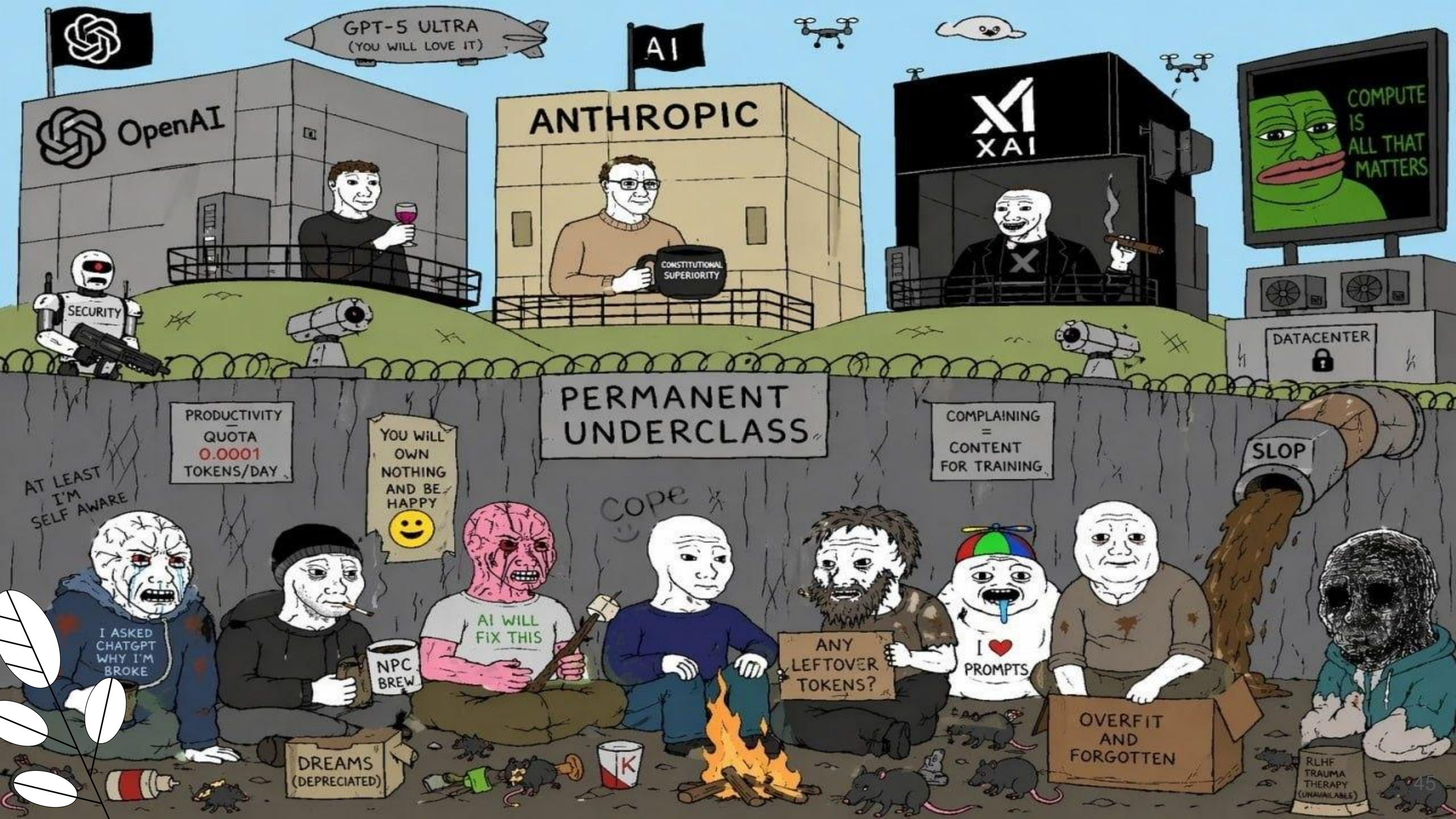
- Несколько запусков в виде непрерывного producer-consumer **цикла**

- Три агента, которые **проверяют** друг друга, голосуют и ранжируют результаты

- **Динамическое моделирование угроз** — агенты строят threat model на основе репозитория, а не получают его заранее

К выводам!



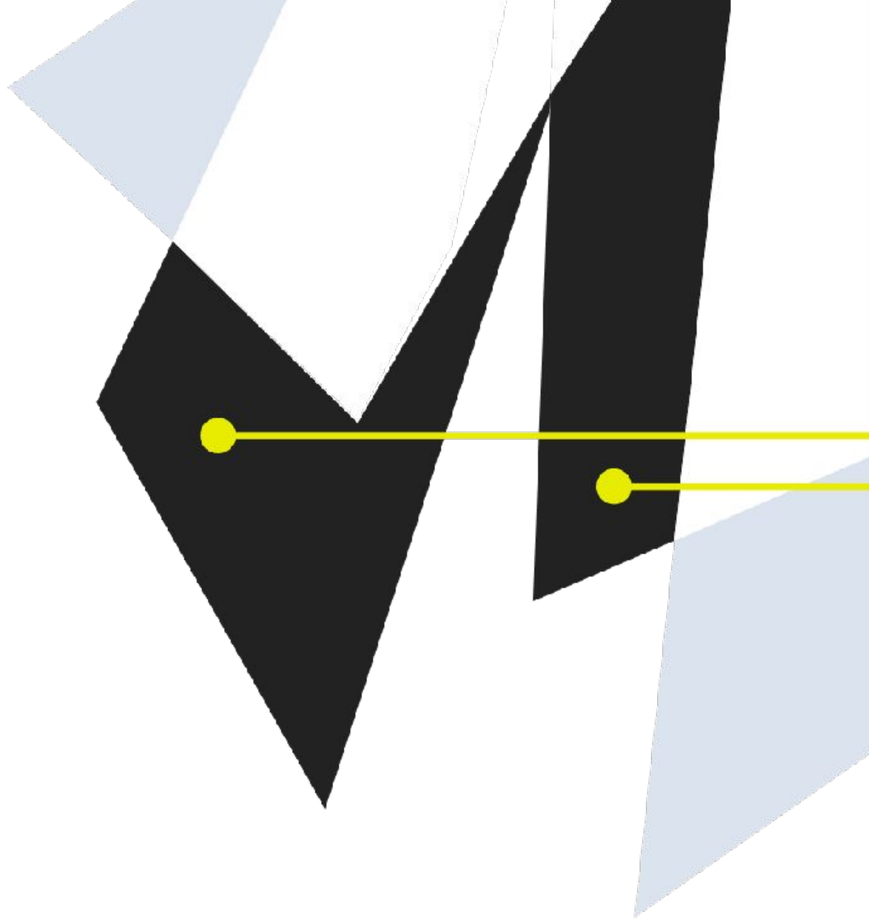

GPT-5 ULTRA
(YOU WILL LOVE IT)
AI
OpenAI
ANTHROPIC
CONSTITUTIONAL SUPERIORITY
XAI
COMPUTE IS ALL THAT MATTERS
SECURITY
DATACENTER
PERMANENT UNDERCLASS
PRODUCTIVITY — QUOTA 0.0001 TOKENS/DAY
YOU WILL OWN NOTHING AND BE HAPPY
COMPLAINING = CONTENT FOR TRAINING
SLOP
AT LEAST I'M SELF AWARE
cope
I ASKED CHATGPT WHY I'M BROKE
NPC BREW
AI WILL FIX THIS
ANY LEFTOVER TOKENS?
I ♥ PROMPTS
OVERFIT AND FORGOTTEN
DREAMS (DEPRECIATED)
RLHF TRAUMA THERAPY (UNAVAILABLE)

SourceCraft

- Мир агентных систем далёк от совершенства

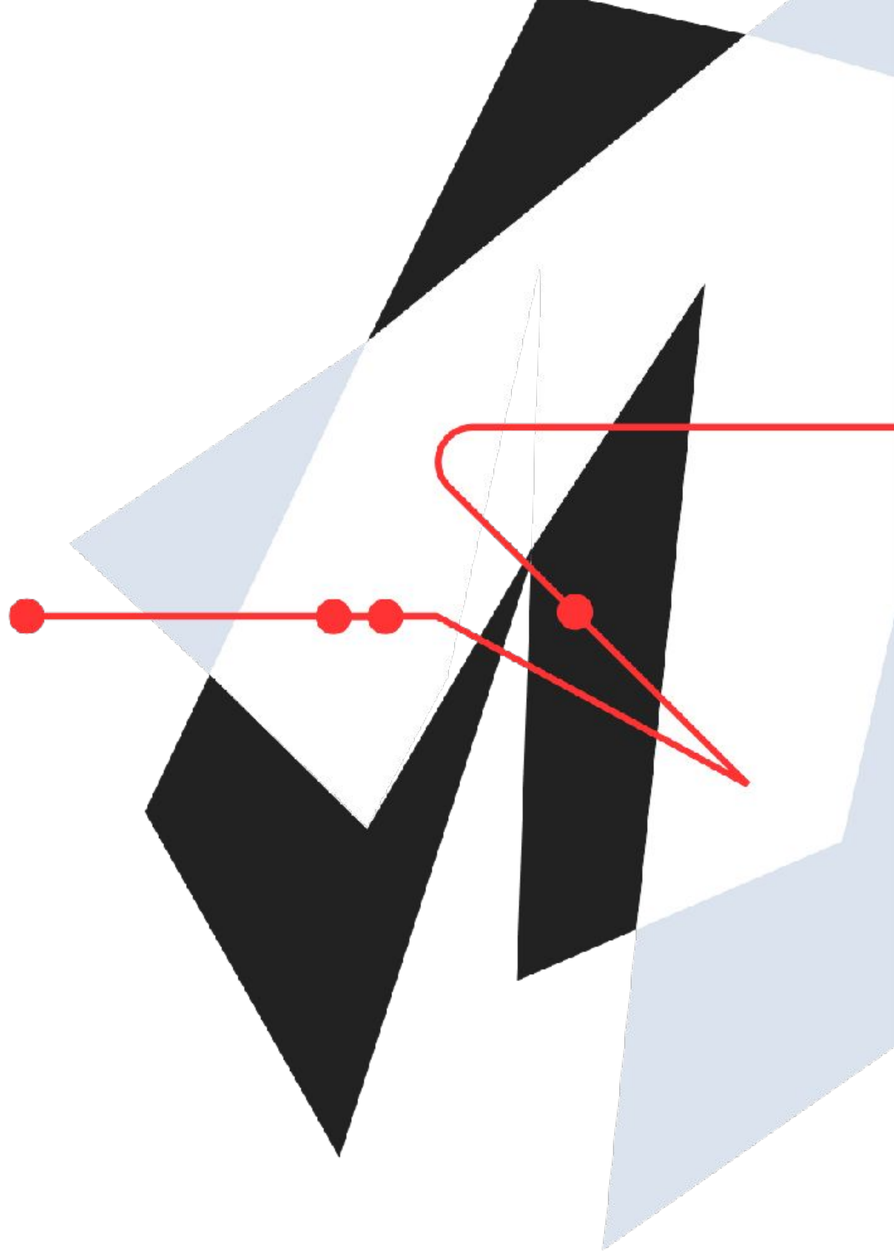
- У традиционного анализа кода тоже есть свои ограничения

- Самый сильный подход — **гибридный**: классический анализ отвечает за факты, ИИ — за контекст и рассуждение

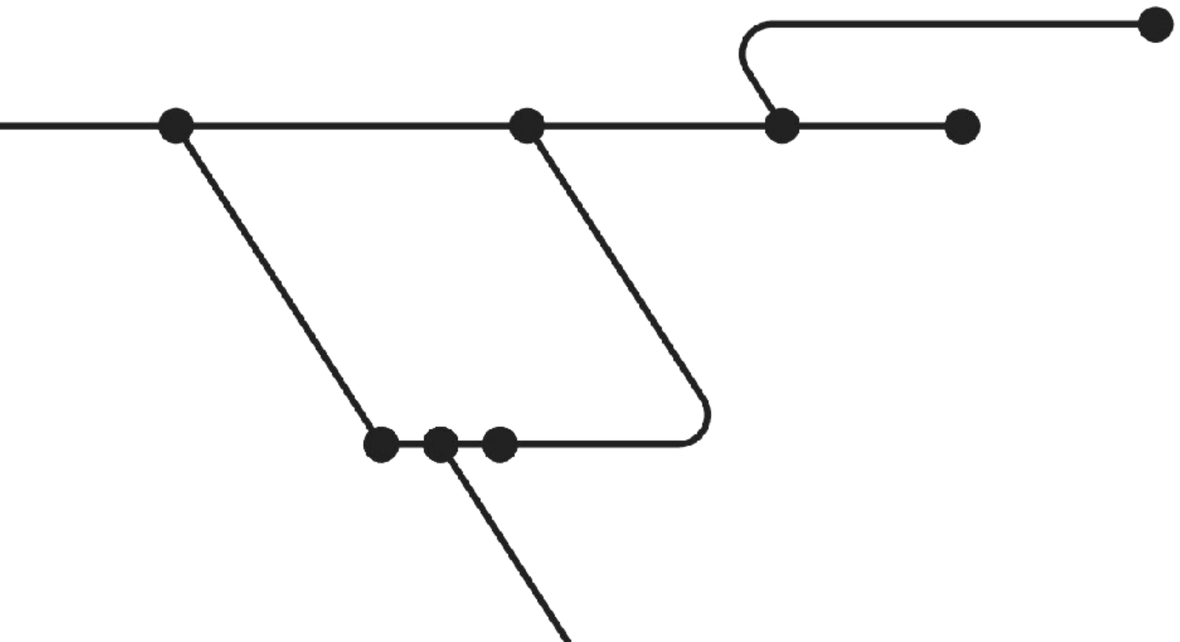


SourceCraft

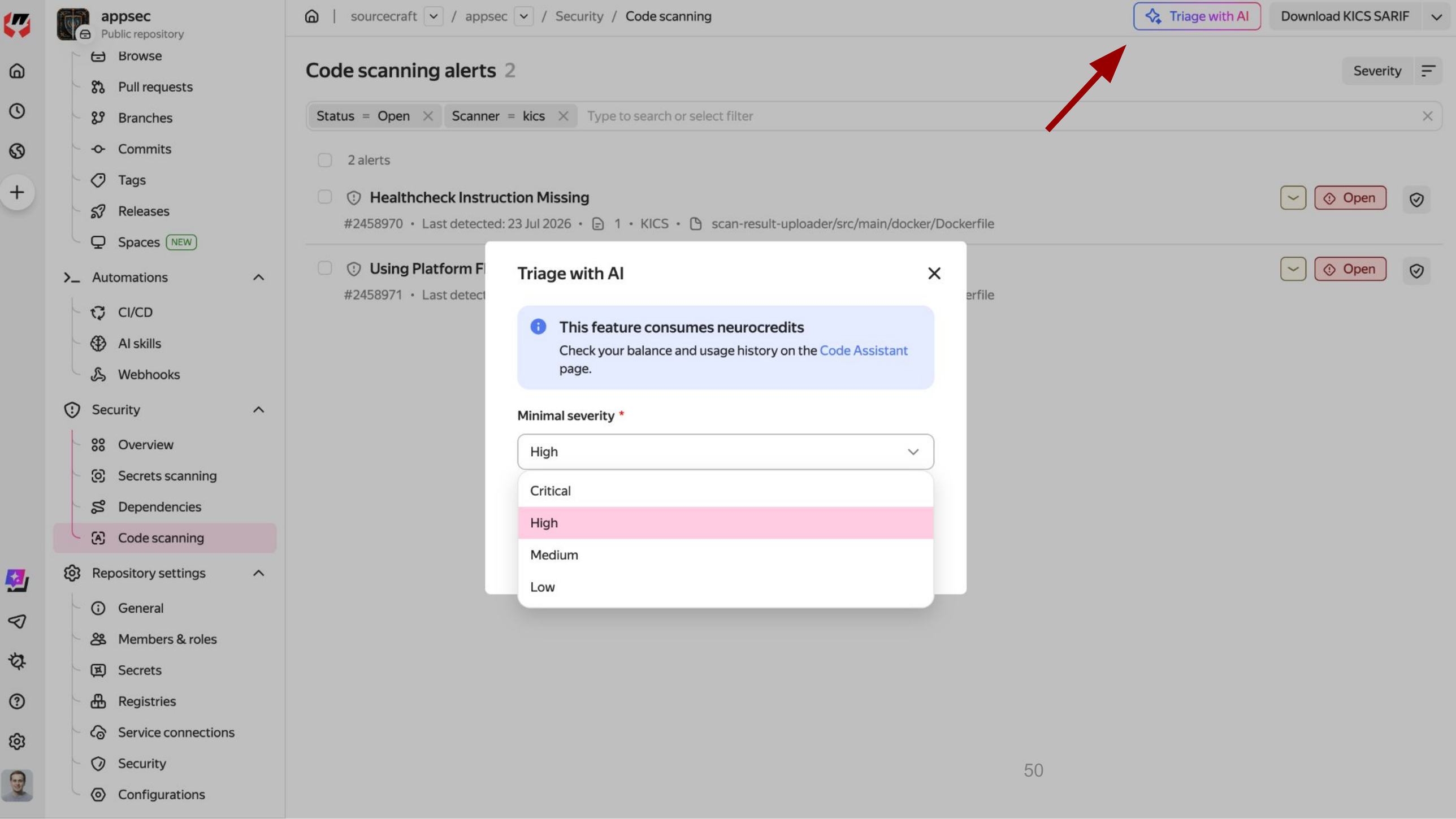
Спасибо!



Пример на SourceCraft



Triage


appsec
Public repository
Browse
Pull requests
Branches
Commits
Tags
Releases
Spaces NEW
Automations
CI/CD
AI skills
Webhooks
Security
Overview
Secrets scanning
Dependencies
Code scanning
Repository settings
General
Members & roles
Secrets
Registries
Service connections
Security
Configurations
sourcecraft / appsec / Security / Code scanning
Triage with AI
Download KICS SARIF
Code scanning alerts 2
Severity
Status = Open
Scanner = kics
Type to search or select filter
2 alerts
Healthcheck Instruction Missing
#2458970 • Last detected: 23 Jul 2026 • 1 • KICS • scan-result-uploader/src/main/docker/Dockerfile
Open
Using Platform F
#2458971 • Last detect
erfile
Open
Triage with AI
This feature consumes neurocredits
Check your balance and usage history on the Code Assistant page.
Minimal severity *
High
Critical
High
Medium
Low

Triage with AI | Download KICS SARIF

appsec
Public repository

- Browse
- Pull requests
- Branches
- Commits
- Tags
- Releases
- Spaces NEW

Automations
- CI/CD
- AI skills
- Webhooks

Security
- Overview
- Secrets scanning
- Dependencies
- Code scanning

Repository settings
- General
- Members & roles
- Secrets
- Registries
- Service connections
- Security
- Configurations

Code scanning alerts 2

Severity

Status = Open | Scanner = kics | Type to search or select filter

2 alerts

Severity from AI: Low

Healthcheck Instruction Missing — Open
#2458970 • Last detected: 23 Jul 2026 • 1 • KICS • scan-result-uploader/src/main/docker/Dockerfile

Using Platform Flag with FROM Command — Open
#2458971 • Last detected: 23 Jul 2026 • 1 • KICS • scan-result-uploader/src/main/docker/Dockerfile



Search

Plan
- Issues
- Milestones
- Labels

Code
- Browse
- Pull requests
- Branches
- Commits
- Tags
- Releases
- Spaces NEW

Automations
- CI/CD
- AI skills
- Webhooks

Security
- Overview
- Secrets scanning
- Dependencies
- Code scanning

Repository settings

Using Platform Flag with FROM Command

Last detected: 23 Jul 2026 • KICS • scan-result-uploader/src/main/docker/Dockerfile:1:0

Overview | Locations 1

Detected locations 1

Show all locations

FROM={{--platform=linux/amd64 registry.access.redhat.com/ubi9/openjdk-25:1.24}}.{{FROM --platform=linux/amd64 registry.access.redhat.com/ubi9/openjdk-25:1.24}} is using the '--platform' flag

scan-result-uploader / src / main / docker / Dockerfile

Edit file

```
1  FROM --platform=linux/amd64 registry.access.redhat.com/ubi9/openjdk-25:1.24
2
3  # Switch to root to install tools and configure system
4  USER 0
```

AX axidex • 17 Jun 2026, 20:24 • 2c67f6

Links 1

https://docs.docker.com/engine/reference/builder/#from

Activity

Add your markup here

Comment

28 May 2026

Overview Locations 1

Andrey Kuleshov Marked the alert as tolerable and closed it. • 23 Aug 2026, 02:23 • Open » Resolved Tolerable

The status has been changed by the AppSec AI neurotriage: the trigger is true, but the risk is acceptable and no action is required.

SourceCraft Security Bot • 23 Aug 2026, 02:23

1. Rating

- Type: True positive (according to the KICS rule), but from a practical standpoint — an acceptable risk.
- Confidence: HIGH
- Justification: The rule correctly detects the use `--platform` in the instruction `FROM`, however, this is standard practice for multi-architecture builds and does not pose a direct security threat.

2. Risk

- Level: LOW
- Exploitability: Not directly exploitable. The flag `--platform` defines the target architecture of the image and does not create an attack vector.
- Impact: None. The attacker cannot use this flag to gain access or cause damage.

3. Correction

Safe code (if you need to remove the flag):

```
# Было:
FROM --platform=linux/amd64 alpine:latest

# Стало:
FROM alpine:latest
```

Key steps:

- If a multi-architecture build is not required — remove the flag `--platform`.
- If support for multiple architectures is required, use `docker buildx` with platform parameters outside the Dockerfile (for example, `--platform` in the build command), keeping the Dockerfile clean.
- Check: run `docker build` and make sure the image is built correctly.

Fix

gitleaks
Public repository

Search

Plan
Issues
Milestones
Labels

Code
Browse
Pull requests
Branches
Commits
Tags
Releases
Spaces NEW

Automations
CI/CD
AI skills
AI Jobs NEW
Webhooks

People & robots
AI Coworkers NEW
Members & roles

Security
Overview

golang.org/x/crypto/ssh: Denial of Service in the Key Exchange of golang.org/x/crypto/ssh

Last detected: 19 Aug 2026 • Trivy • golang.org/x/crypto@v0.32.0 • Transitive

CVE-2025-22869

Reopen | Start fixing | Triage Problem

SSH servers which implement file transfer protocols are vulnerable to a denial of service attack from clients which complete the key exchange slowly, or not at all, causing pending content to be read into memory, but never transmitted.

Direct dependencies from the project 1

For mitigating this vulnerability you need to update this packages with package manager for your programming language.

github.com/sagikazarmark/locafero v0.7.0

Links 6

https://avd.aquasec.com/nvd/cve-2025-22869
https://nvd.nist.gov/vuln/detail/cve-2025-22869
https://moshub.ru
#1837 False positive: e.unk_token,this.vocab=new — Open
!1834 feat(rules): Add 1Password secret key detection — Open
!1808 Add Tailscale key rules — Open

Alert details

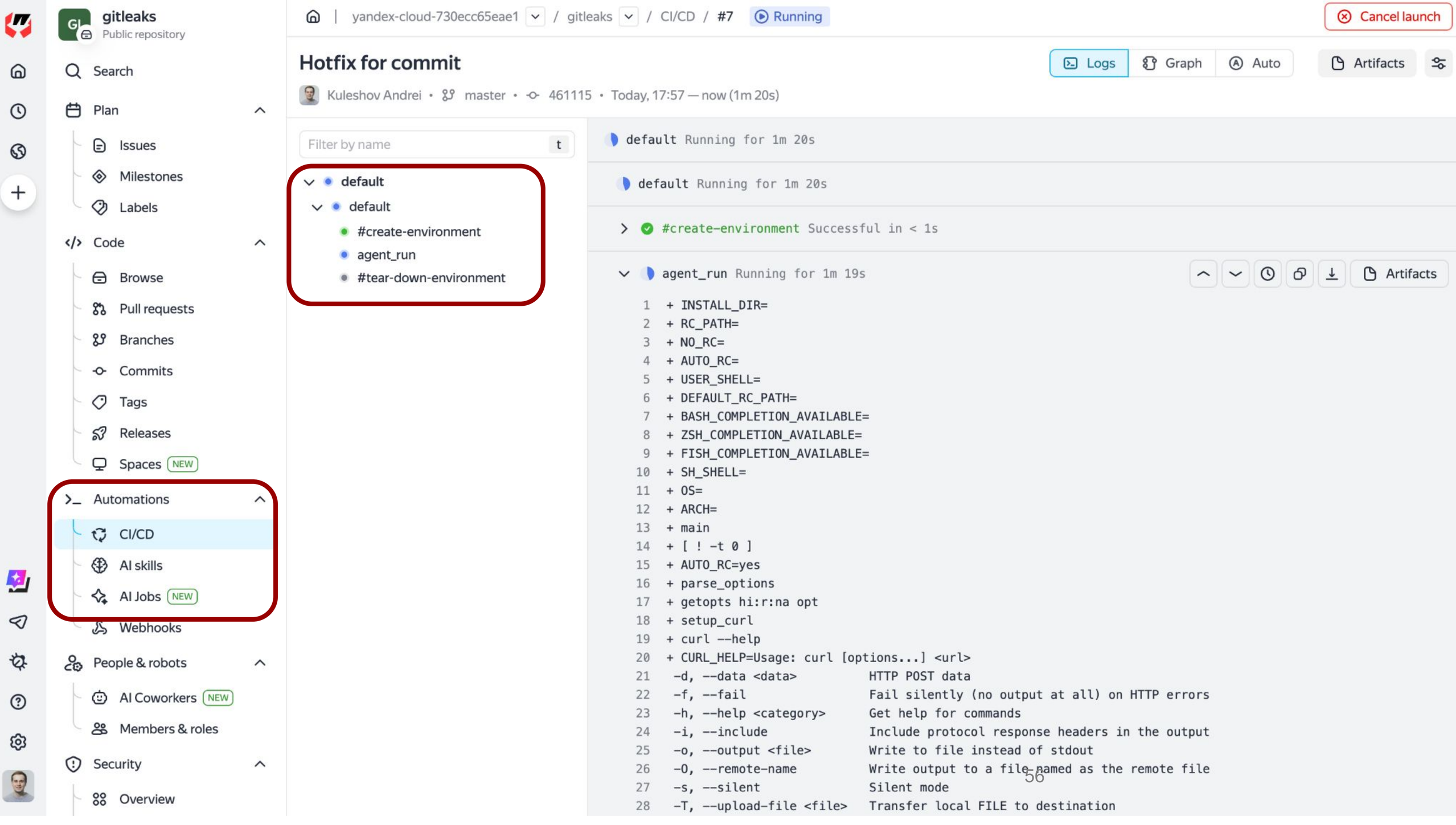
CVE-2025-22869
CVSS: 8.0 (High)
Last detected: 19 Aug 2026

Repository

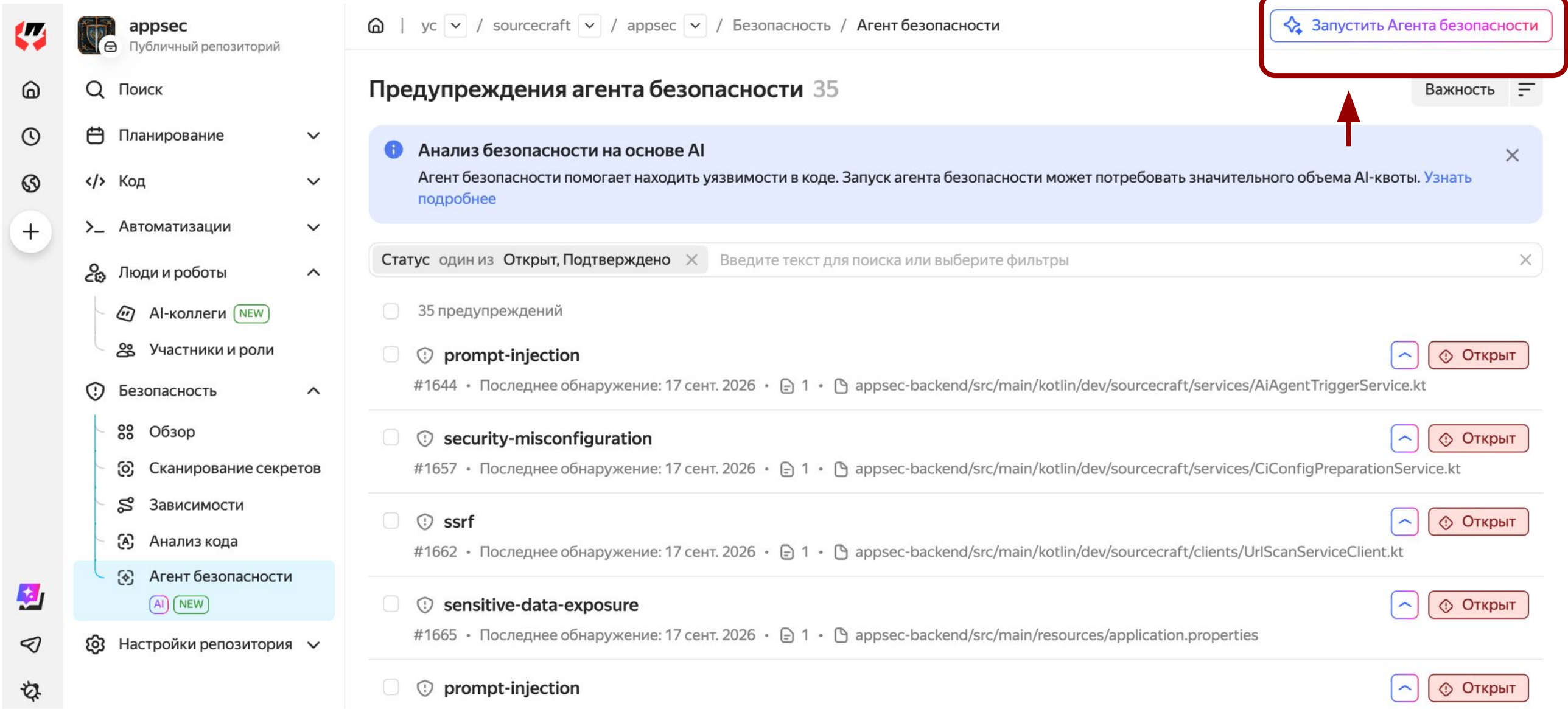
yandex-cloud-730ecc65eae1/gitleaks
master
4611151

Package

golang.org/x/crypto
0.35.0
Patched version


gitleaks
Public repository
Search
Plan
Issues
Milestones
Labels
Code
Browse
Pull requests
Branches
Commits
Tags
Releases
Spaces NEW
Automations
CI/CD
AI skills
AI Jobs NEW
Webhooks
People & robots
AI Coworkers NEW
Members & roles
Security
Overview
yandex-cloud-730ecc65eae1 / gitleaks / CI/CD / #7 Running
Cancel launch
Hotfix for commit
Logs
Graph
Auto
Artifacts
Kuleshov Andrei • master • 461115 • Today, 17:57 — now (1m 20s)
Filter by name
default
default
#create-environment
agent_run
#tear-down-environment
default Running for 1m 20s
default Running for 1m 20s
#create-environment Successful in < 1s
agent_run Running for 1m 19s
Artifacts
+ INSTALL_DIR=
+ RC_PATH=
+ NO_RC=
+ AUTO_RC=
+ USER_SHELL=
+ DEFAULT_RC_PATH=
+ BASH_COMPLETION_AVAILABLE=
+ ZSH_COMPLETION_AVAILABLE=
+ FISH_COMPLETION_AVAILABLE=
+ SH_SHELL=
+ OS=
+ ARCH=
+ main
+ [! -t 0]
+ AUTO_RC=yes
+ parse_options
+ getopts hi:r:na opt
+ setup_curl
+ curl --help
+ CURL_HELP=Usage: curl [options...] <url>
-d, --data <data> HTTP POST data
-f, --fail Fail silently (no output at all) on HTTP errors
-h, --help <category> Get help for commands
-i, --include Include protocol response headers in the output
-o, --output <file> Write to file instead of stdout
-O, --remote-name Write output to a file named as the remote file
-s, --silent Silent mode
-T, --upload-file <file> Transfer local FILE to destination

Full-audit


appsec
Публичный репозиторий
Поиск
Планирование
Код
Автоматизации
Люди и роботы
AI-коллеги NEW
Участники и роли
Безопасность
Обзор
Сканирование секретов
Зависимости
Анализ кода
Агент безопасности AI NEW
Настройки репозитория
yc / sourcecraft / appsec / Безопасность / Агент безопасности
Запустить Агента безопасности
Предупреждения агента безопасности 35
Важность
Анализ безопасности на основе AI
Агент безопасности помогает находить уязвимости в коде. Запуск агента безопасности может потребовать значительного объема AI-квоты. Узнать подробнее
Статус один из Открыт, Подтверждено
Введите текст для поиска или выберите фильтры
35 предупреждений
prompt-injection
#1644 • Последнее обнаружение: 17 сент. 2026 • 1 • appsec-backend/src/main/kotlin/dev/sourcecraft/services/AiAgentTriggerService.kt
Открыт
security-misconfiguration
#1657 • Последнее обнаружение: 17 сент. 2026 • 1 • appsec-backend/src/main/kotlin/dev/sourcecraft/services/CiConfigPreparationService.kt
Открыт
ssrf
#1662 • Последнее обнаружение: 17 сент. 2026 • 1 • appsec-backend/src/main/kotlin/dev/sourcecraft/clients/UrlScanServiceClient.kt
Открыт
sensitive-data-exposure
#1665 • Последнее обнаружение: 17 сент. 2026 • 1 • appsec-backend/src/main/resources/application.properties
Открыт
prompt-injection
Открыт

SourceCraft

Спасибо!

